



Technical Reference

Modules, APIs and code review

Prepared by – **iorta TechNXT**

Version - **1.1**

Date - **04 October 2026**

TABLE OF CONTENTS

1. Document Control.....	5
2. Acronyms.....	5
3. Introduction.....	7
3.1 Purpose and readers.....	7
3.2 Sources and how the figures were measured.....	7
3.3 Companion workbook.....	8
4. Solution layout.....	9
4.1 Repository structure.....	9
4.2 Back-end start-up and request pipeline.....	9
4.3 Back-end module pattern.....	10
4.4 Shared libraries.....	11
4.5 Database, migrations and seeds.....	12
4.6 Front-end structure.....	13
4.7 How a request flows from a screen to the database and back.....	13
5. Module catalogue.....	15
5.1 How to read the catalogue.....	15
5.2 Sales, placement and policy.....	15
5.3 Distribution, motor and specialty lines.....	16
5.4 Claims.....	17
5.5 Accounting and finance.....	17
5.6 Compliance.....	19
5.7 Integrations.....	19
5.8 Platform, configuration and administration.....	20
5.9 Scheduled jobs by module.....	21
6. Platform engines.....	23
6.1 Route registry and API conventions.....	23
6.2 Permissions and roles.....	24
6.3 Posting rules engine.....	24
6.4 Tax engine.....	25
6.5 Document numbering.....	26
6.6 Scheduled jobs and the scheduler.....	26
6.7 PDF, Excel and e-mail engines with branding.....	26
6.8 E-signatures.....	27
6.9 Integration framework.....	27
6.10 Encryption and masking of personal data.....	29
6.11 Audit trail.....	30

6.12 Go-live workbench and data tools.....	30
6.13 Release pipeline.....	31
7. Coding standards followed in this code base.....	32
7.1 Naming and file layout.....	32
7.2 Validation with zod.....	32
7.3 Errors and the response envelope.....	32
7.4 SQL: parameters, pool and transactions.....	33
7.5 Configuration through app_settings.....	33
7.6 Secrets and environment.....	33
7.7 Logging.....	34
7.8 Front-end conventions.....	34
8. Code quality controls.....	35
8.1 Size of the code base.....	35
8.2 Static checks.....	35
8.3 Automated tests.....	35
8.4 Consistency checks.....	36
8.5 Migration discipline.....	36
8.6 UAT scenario script.....	37
8.7 Continuous integration.....	37
9. Security controls in code.....	38
9.1 Authentication.....	38
9.2 Password policy, lockout and two-factor sign-in.....	38
9.3 Authorisation.....	38
9.4 SQL injection prevention.....	39
9.5 Input validation and request limits.....	40
9.6 HTTP headers, CORS and rate limiting.....	40
9.7 File uploads and downloads.....	40
9.8 Encryption and secrets.....	41
9.9 Audit and sign-in logging.....	41
9.10 OWASP Top 10 (2021) mapping.....	41
9.11 Points for the security review before go-live.....	42
10. Front end to back end dependencies.....	43
10.1 How the matrix was built.....	43
10.2 Summary per menu area.....	43
10.3 Prerequisites per back-end module.....	45
10.4 Set-up order used by the UAT scenario.....	46
10.5 Before enhancing a module.....	46
11. API catalogue summary.....	47

11.1 Endpoints per module.....	47
11.2 Public endpoints.....	48
11.3 Keeping the API documentation current.....	49
12. How to review a change.....	50
12.1 What the pull request must contain.....	50
12.2 Code standards to check.....	50
12.3 Test expectations.....	50
12.4 Table classification and personal data catalogue.....	51
12.5 Migration numbering.....	51
12.6 Translations, menu, grants and help.....	52
12.7 Lint and checks to run.....	52
12.8 CI gates.....	52
13. Guide for the production support team.....	53
13.1 Running locally.....	53
13.2 Where the logs are.....	53
13.3 Tracing a defect from a screen to the SQL.....	53
13.4 Common failure patterns and where to look.....	54
13.5 Making a change safely.....	55
13.6 Release and rollback.....	56
14. Document Sign-Off.....	57

1. Document Control

Date	Version	Author & Business Unit Info	Reviewed By & Business Unit Info	Approved By & Business Unit Info	Change History
04 October 2026	1.1.1	iorta TechNXT			Version 1.1.1: release figures aligned (packages B and G merged: 73 modules, 1,356 endpoints, migrations to 0331, 104 test files with 1,113 tests, 33 front-end suites with 175 tests). Version 1.1: module catalogue of all 70 modules; platform engines (registry, permissions, posting, tax, numbering, jobs, PDF, Excel and e-mail with branding, e-signatures, integrations, encryption and masking, audit, go-live workbench, release pipeline); how to review a change; figures at migration 0322

2. Acronyms

Acronym / Symbol	Definition / Meaning
OOTB	Out of the box
API	Application programming interface
JWT	JSON Web Token
SQL	Structured Query Language
ESM	ECMAScript modules
CRA	Create React App

GL	General ledger
LGU	Local government unit
LGT	Local government tax
DST	Documentary stamp tax
FST	Fire service tax
VAT	Value-added tax
EWT	Expanded withholding tax
BIR	Bureau of Internal Revenue
EOPT	Ease of Paying Taxes Act
EIS	Electronic Invoicing System
CAS	Computerized accounting system
IC	Insurance Commission
NPC	National Privacy Commission
AML	Anti-money laundering
CFT	Countering the financing of terrorism
AMLC	Anti-Money Laundering Council
EDD	Enhanced due diligence
KYC	Know your customer
CTPL	Compulsory third party liability
COC	Certificate of cover
LTO	Land Transportation Office
PDC	Post-dated cheque
PSGC	Philippine Standard Geographic Code
PII	Personally identifiable information
TOTP	Time-based one-time password
HMAC	Hash-based message authentication code
AES	Advanced Encryption Standard
CORS	Cross-origin resource sharing
OWASP	Open Worldwide Application Security Project
UAT	User acceptance testing
SIT	System integration testing

CI	Continuous integration
CAB	Change advisory board
PM2	Node.js process manager used on the EC2 server

3. Introduction

3.1 Purpose and readers

This document describes how BrokerVerse OOTB is built: where the code is, what each back-end module does, the shared engines every module relies on, the conventions the code follows, the controls that keep it correct and secure, which screen depends on which API, how a change is reviewed and how the production support team traces and changes the system. It is written for the developers who enhance the product, for the code reviewers who approve their pull requests and for the support team that takes the system over after go-live.

The business use of each screen is in the User Manual. Deployment steps are in `deploy/README.md`, `deploy/REFERENCE.md` and `deploy/RELEASE_PIPELINE.md` of the repository. This document points to them where they apply and does not repeat them.

3.2 Sources and how the figures were measured

Every statement in this document comes from the repository as it stood on 04 October 2026: branch `brokerverse-platform` with packages B (Insurance Commission and data privacy compliance) and G (sales activities, quote covers and risk fields, supplier BIR Form 2307 and fixed asset disposal) merged into it on 04 October 2026; the packages are still named where a feature came with them.

Figure	How it was measured
Endpoints, permissions, screens	Route registry loaded by <code>collectRoutes()</code> of <code>backend/src/tools/export-api.js</code> (the loader of <code>npm run export:api</code>), on the branch and on each package branch
Lines of code	Line count of <code>.js</code> , <code>.jsx</code> , <code>.sql</code> , <code>.json</code> (seeds) and <code>.scss</code> files, <code>node_modules</code> excluded

Figure	How it was measured
Tables, settings, roles, permissions, jobs	A new database built on 04 October 2026 with the migrations (and those of packages B and G) and the reference seed data only (SEED_SAMPLE_DATA=false)
Back-end tests	npm test (vitest) on 04 October 2026 against a new test database
Front-end tests	CI=true npx craco test --watchAll=false on 04 October 2026
Lint	npm run lint in backend/ and in brokerverse/ on 04 October 2026
Screen dependencies	Menu tree, routes and the import closure of every screen, matched against the route registry (chapter 9)

3.3 Companion workbook

The full reference lists are in the workbook

BrokerVerse_API_and_Dependency_Catalogue.xlsx, delivered with this document and regenerated on 04 October 2026:

- **APIs:** every endpoint with its back-end module, method, path, summary, permission and screen (1,291 rows).
- **Screen dependencies:** every menu screen with its route, front-end file, service files, the endpoints it calls, the back-end modules behind them and the data that must exist first (223 rows).
- **Summary:** endpoints per back-end module by method, the number of menu screens that use each module, and the prerequisites per module (70 modules).

The column-level description of the database is in the Data Dictionary and its workbook.

4. Solution layout

4.1 Repository structure

BrokerVerse is one repository with two applications and their deployment files.

Folder	Content
backend/	REST API and scheduled jobs: Node.js 22, ECMAScript modules, Express 4, PostgreSQL through pg
backend/src/modules/	73 business modules, one folder each (router, service, SQL)
backend/src/lib/	Shared libraries: registry, authentication, errors, validation, settings, audit, logging, PDF, Excel, e-mail, numbering, addresses, currency
backend/src/db/	Connection pool, migration runner, seed runner, 151 migrations (0001 to 0331), reference and sample seeds
backend/src/jobs/	Cron scheduler, job handlers, housekeeping
backend/test/	vitest suite (104 test files) with helpers and fixtures
backend/scripts/	Settings check, UAT scenario, go-live rehearsal, transaction reset, sample-data purge, data masking, environment comparison, brand packs, geography loaders, upload templates
backend/docs/	API documentation output (OpenAPI, Postman, Excel touchpoints)
brokerverse/	Web application: React 18 built with Create React App and craco
brokerverse/src/	Screens (module/, agentModule/), services, routes, menu, theme, runtime branding, translations, help panel
deploy/	Runtime reference, release pipeline, EC2 release script, PM2 and nginx configuration, environment examples, smoke test
docker-compose.yml	PostgreSQL 16, API and web on one server (test system or trial)
.github/workflows/	CI and release artefacts (ci.yml), deployment to an environment (deploy.yml), rollback (rollback.yml)
docs/	Documentation pack (docs/package), architecture set (docs/architecture), developer guide, onboarding guides

Main run-time libraries of the back end: express 4.21, pg 8.13, zod 3.24, jsonwebtoken 9, bcryptjs 2.4, helmet 8, cors 2.8, multer 1.4, pino 9 with pino-http 10, node-cron 4, nodemailer 10. PDF and Excel files are written by the platform's own code (lib/pdf, lib/xlsx.js) without further libraries. Main libraries of the front end: react 18.2, react-router-dom 6.30, primereact 10.3 with primereflex, @reduxjs/toolkit 2, axios 1.20, formik 2, i18next 25 with react-i18next, chart.js 4, moment.

4.2 Back-end start-up and request pipeline

backend/src/server.js starts the API in this order:

1. Refuse to start in production with an unsafe configuration (assertProductionConfig() in src/config.js).
2. Apply pending migrations (src/db/migrate.js), one instance at a time under a PostgreSQL advisory lock.
3. Run the idempotent seed (src/db/seed.js): roles, permissions, administrator, settings, scheduled jobs, reference data.
4. Build the Express application (createApp() in src/app.js) and listen on PORT.
5. Start the cron scheduler and the watch for changed schedules, then report ready on GET /api/health.

`createApp()` sets up the middleware in a fixed order. Every request passes through it before it reaches a module.

Order	Middleware	Source	What it does
1	<code>requestId</code>	<code>src/app.js</code>	Takes a well-formed x-request-id or creates a UUID; echoes it on the response
2	<code>requestContext</code>	<code>src/lib/requestContext.js</code>	Keeps the request in AsyncLocalStorage (current user for document builders and exports)
3	<code>helmet</code>	<code>helmet</code>	Security headers
4	<code>cors</code>	<code>cors</code>	Origins from <code>CORS_ORIGINS</code> ; exposes x-request-id, Retry-After, Content-Disposition
5	<code>pinoHttp</code>	<code>src/lib/logger.js</code>	One JSON log line per request with the request id; secrets redacted
6	<code>apiRateLimit</code>	<code>src/lib/rateLimit.js</code>	Per user (valid token) or per IP; setting <code>security.api_rate_limit</code>
7	<code>express.json</code>	<code>express</code>	Body limit <code>JSON_BODY_LIMIT</code> (2mb); keeps the raw body for payment and integration webhooks
8	<code>signFileLinks</code>	<code>src/lib/fileLinks.js</code>	Signs stored-file links in responses
9	<code>piiMiddleware</code> , <code>complianceWarningsMiddleware</code> are (package B)	<code>src/lib/piiPolicy.js</code> , <code>src/lib/complianceWarnings.js</code>	Decrypts personal identifiers on the way out and masks them for users without <code>view:pii</code> ; carries compliance warnings (insurer authority, referrer licence) with the answer
10	Module routers	<code>src/modules/*/router.js</code>	Mounted under <code>/api</code> by <code>loadModules()</code>
11	Not-found and <code>errorHandler</code>	<code>src/lib/errors.js</code>	One JSON error envelope for every failure

4.3 Back-end module pattern

Every folder under `backend/src/modules` that holds a `router.js` is mounted under `/api` automatically. A module exports its router as default, and may export `mount` (path prefix), `extraMounts` (further routers at their own prefix) and `order`. The files of a module follow one pattern.

File	Role
<code>router.js</code>	Declares every endpoint with <code>define()</code> : method, path, summary, screen, permission middleware, zod validation, example request and response, handler
<code>service.js</code>	Business rules and SQL; receives a database client (db) when it must run inside a transaction
Named files (<code>billing.js</code> , <code>placements.js</code> , <code>assignment.js</code> , <code>licences.js</code> ...)	Larger modules split their service by topic
<code>jobs.js</code>	Scheduled job handlers of the module, re-exported by <code>src/jobs/handlers.js</code>
<code>README.md</code> (some modules)	Module notes: accounting, period-end, reports,

File	Role
	document numbering, posting rules

A route declaration from the receipts module

(backend/src/modules/receipts/router.js):

```
define({
  method: 'POST', path: '/', summary: 'Finance (write:receipts): create an official receipt ...',
  screen: `${SCREEN} > Add receipt`, middleware: [...write, validate(receiptSchema)],
  request: { receiptType: 'Payment', receiptDate: '2026-09-28T00:00:00.000Z', ... },
  response: { success: true, data: example },
  handler: async (req, res) => {
    const r = await withTransaction((db) => svc.createReceipt(db, req.body, req.user));
    await audit(req, { entity: 'receipt', entityId: r.receiptId, action: 'create', after: r });
    created(res, r, `Receipt ${r.receiptNumber} created`);
  },
});
```

write is [requireAuth, requirePermission('write:receipts')], declared once at the top of the router. The handler holds no SQL: the service does the work inside one transaction, the router writes the audit record and the response.

4.4 Shared libraries

File in backend/src/lib	Used for
registry.js	moduleRouter() and define(): one declaration feeds Express, the API list, OpenAPI and Postman
auth.js	Tokens, requireAuth, requirePermission, requireRole, isAdmin, hasPermission, session revocation
errors.js	HttpError, badRequest, unauthorized, forbidden, notFound, conflict, errorHandler
validate.js	validate(schema, part) middleware over zod
respond.js	ok(), created(), paging(), pageMeta(), wrap()
settings.js, settingOwners.js	getSetting(), getSettings(), setSetting() over app_settings, with a cache; keys owned by a screen of their own are refused by the generic endpoints
audit.js, auditEvents.js, auditLabels.js	audit(req, {...}) into audit_log with its source; the audit trail read back as business events with labels
scope.js	Record-level scoping of users whose roles are all in security.scoped_roles
makerChecker.js	Approver must differ from the maker (finance.maker_checker_enabled)
password.js, totp.js, secrets.js	Password policy, two-factor codes, encryption of two-factor secrets, signed links
pii.js, piiPolicy.js (package B)	Field encryption of personal identifiers, masking by role, key ring and rotation
rateLimit.js, uploadLimits.js	Sign-in and API limiters; upload, ZIP inflation and import row caps
numbering.js	nextDocumentNumber() over the document_numbering series
logger.js, loginHistory.js, health.js	Logging with redaction, sign-in history, readiness and liveness

File in backend/src/lib	Used for
pdf/, letterhead.js, template.js	The PDF engine with the company letterhead and signatures; {{placeholders}} in templates
xlsx.js, csv.js, zip.js	Branded Excel and CSV output; reading of uploaded workbooks
mailer.js, publicWeb.js	E-mail through SMTP_URL with the branded layout; public web address for links
money.js, currency.js, dates.js, birthDate.js	Rounding; base currency and dated exchange rates; business dates in the configured time zone
address.js	Philippine address format and region derived from the city or province
environment.js, goLiveLock.js	Environment marker (system.environment) and go-live lock (goLive.locked) read by the data tools

4.5 Database, migrations and seeds

The database is PostgreSQL 16 (the version in docker-compose.yml and in CI). A database migrated and seeded on 04 October 2026 with the reference data and the migrations of packages B and G holds 260 tables and 2 views, 73 functions, 660 settings in 64 groups (659 before migration 0330), 8 roles, 98 permissions, 69 master types, 40 report definitions, 85 document number series, 35 scheduled jobs and 43 posting events with an active rule. On the branch alone (without packages B and G) it holds 250 tables.

- **Migrations** (backend/src/db/migrations, 151 files from 0001_core.sql to 0331_pii_client_identifiers.sql; package B brought 0270 to 0277, package G 0320 to 0322, and the release verification of 04 October 2026 added 0330, the EIS outbox restart safety setting, and 0331, the encryption of the client identifiers of the AML/CFT onboarding; 0323 to 0329 are unused) run in file-name order. Each runs in its own transaction and is recorded in schema_migrations. Applied migrations are never edited; a correction is a new file.
- **Seeds** (backend/src/db/seeds) run on every start and are idempotent: rows are inserted by natural key and existing rows and administrator edits are kept. settings.json holds configuration keys, jobs.json the scheduled jobs, the numbered SQL files the reference data: masters (10_, 51_), the Philippine geography from the PSGC of the 2nd quarter 2026 (12_ph_geography.sql), chart of accounts, motor tariff, product templates and rules (52_, 53_), report catalogue, security, Philippine practice masters such as banks, ID types, salutations, holidays and the IC insurer list (69_ph_practice_masters.sql), BIR forms (72_), operations and accounting rules (73_) and integration connectors (75_). seeds/sample/ holds demo data, loaded only when SEED_SAMPLE_DATA is on.
- **Connection pool** (backend/src/db/pool.js): one pg pool of 10 connections; query(), one(), many(), withTransaction(fn) and runInTransaction() (an ambient transaction used by the go-live validation). NUMERIC and BIGINT values come back as numbers and DATE values as YYYY-MM-DD strings. With package B every connection carries the personal data keys as session settings.

4.6 Front-end structure

Path in brokerverse/src	Content
index.js, App.js	Entry point; store, router, theme, session renewal; sign-in and public routes (/login, /approve-quote)
routes/MainRoute.js	Every signed-in route (473 Route elements), wrapped by routes/ProtectedRoute
components/SideBar/list.js	The side menu as one tree; 223 menu screens; Master in sections (Organization, Insurance Management, Location, Employee Management, User Management, Finance, System Configuration, Data Privacy, Go-Live and Data)
utils/menuPermissions.js	roleMenuPermissions: which role sees which menu entry; route guard isPathAllowed
utils/canOpen.js	canOpen(path) and hasPermission(code) to hide links and actions
components/HelpPanel/	Help panel (F1): helpRoutes.js maps a screen address to its user manual section in public/help/
theme/runtime/	Runtime theme engine and BrandingProvider: the saved theme becomes CSS custom properties without a rebuild
module/	Back-office screens: accounts, finance, masters, placement, remittance, reinsurance, commission, incentive, product configurator, compliance, BIR tax, operations accounting, distribution, integrations, branding, My Work, reports
agentModule/	Operations screens: leads, clients, quotations, policies, endorsements, claims, renewals, payments, open items
services/	57 files, one per API area; all HTTP calls of the screens go through them
utility/	API client (interceptor.js, commonServices.js), token handling, session refresh, idle sign-out, formatting
redux/	Store and reducers (Redux Toolkit)
locales/	en.json (10,578 keys) and th.json
theme/bdoi/	PrimeReact theme and application styles (tokens.scss, bdoi.scss, enterprise.scss)

The front end reaches the API in two ways. Most services call fetch with the bearer header from authService.getAuthHeader(); others use the axios instance in utility/interceptor.js through getRequest, postRequest, putRequest, patchRequest and deleteRequest. Both paths renew an expired access token once: axios through its response interceptor, fetch through the wrapper installed by utility/sessionRefresh.js. The API base address comes from config/runtimeConfig.js: /env-config.js at run time, else REACT_APP_BASE_URL of the build, else the same-origin /api. Lists show skeleton rows while loading (components/DataTable).

4.7 How a request flows from a screen to the database and back

The Add receipt screen saving a receipt shows the full path.

1. The user opens Accounts > Receipts. list.js gives the menu entry and its route /accounts/receipts; ProtectedRoute checks the path against the user's roles with isPathAllowed (deny by default).
2. The screen component calls receiptsService in brokerverse/src/services/receiptsService.js, which sends POST {base}/receipts with the bearer token and JSON body.

3. Express assigns the request id, applies the security headers, CORS, logging and the API rate limit, and parses the JSON body.
4. The receipts router runs `requireAuth` (signature, token type, user active, token version), `requirePermission('write:receipts')` and `validate(receiptSchema)`. A failure ends here with 401, 403 or 400.
5. The handler opens a transaction with `withTransaction()`; `svc.createReceipt(db, ...)` takes a receipt number from the numbering series, writes the receipt and its lines, applies the payment to the open bill and posts the journal through the posting rule `receipt.apply`. All statements use \$1, \$2 ... parameters. Any error rolls the whole transaction back.
6. After the commit the handler writes `audit_log` and answers 201 { success: true, message, data } with the x-request-id header.
7. Any thrown error reaches `errorHandler`: a 4xx keeps its message, a 5xx answers a generic message with the request id and the detail goes to the log.
8. The screen shows the message and returns to the list.

5. Module catalogue

5.1 How to read the catalogue

Every folder under `backend/src/modules` with a `router.js` is one back-end module. The tables below list each module with its purpose, its base paths under `/api` and number of endpoints, the main tables it writes, and the scheduled jobs, posting events and setting groups it uses. Facts come from the route registry, the SQL in the module's files, `backend/src/jobs/handlers.js`, the `posting_rules` rows of a seeded database and the `getSetting()` calls. The per-endpoint list is in the workbook, sheet APIs.

- 73 modules on branch `brokerverse-platform` with packages B and G merged, 1,356 endpoints (70 modules and 1,291 endpoints before the two packages).
- Package B (Insurance Commission and data privacy compliance, merged on 04 October 2026) added the modules `ic-compliance` (38 endpoints) and `data-breaches` (11 endpoints) and changed `clients`, `commission`, `disbursements`, `policies`, `quotations`, `placement`, `privacy` and `search` for the licence block, the insurer authority check and masking.
- Package G (merged on 04 October 2026) added the module `sales-activities` (8 endpoints) and extended `fixed-assets` (disposal, 6 endpoints), `period-end` (supplier BIR Form 2307) and `product-configurator` (covers and risk fields offered in the quote wizard).

5.2 Sales, placement and policy

Module and routes (endpoints)	Purpose	Main tables	Jobs, posting events, settings
<code>leads</code> <code>/leads</code> , <code>/lead</code> , <code>/lead-assignment</code> (21)	Prospects and lead assignment: rules, queue, reassignment, SLA	<code>leads</code> , <code>lead_assignment_rules</code> , <code>lead_assignment_history</code>	Job <code>lead-assignment-sla</code> (off); <code>leads.*</code>
<code>clients</code> <code>/clients</code> , <code>/customers</code> (8)	Clients (individual and juridical), conversion from a lead	<code>clients</code>	KYC fields checked through <code>policy.kyc_*</code>
<code>quotations</code> <code>/quotations</code> , <code>/quote</code> , <code>/master</code> , <code>/email</code> , <code>cover look-ups</code> (43)	Quotations, customer approval link, customer responses, quotation masters and cover look-ups	<code>quotes</code> , <code>quote_customer_responses</code>	Job <code>quote-expiry</code> ; <code>quotations.*</code> , <code>limits.quote_validity_days</code> , <code>premium.*</code>
<code>placement</code> <code>/broker-slips</code> , <code>/placements</code> (26)	Broker slips to several insurers, insurer offers, placement slips, binding per participant	<code>broker_slips</code> , <code>insurer_offers</code> , <code>placements</code> , <code>risk_participants</code>	<code>placement.*</code> , <code>broker_slips.*</code>
<code>comparison-reports</code> <code>/comparison-reports</code> (8)	Client comparison and recommendation report	<code>comparison_reports</code>	<code>comparison.*</code>
<code>cover-notes</code> <code>/cover-notes</code> (7)	Cover notes (binders) while the policy is pending	<code>cover_notes</code>	Job <code>cover-note-expiry</code> ; <code>cover_note.*</code>
<code>policies</code> <code>/policies</code> (12)	Policy issue, KYC on issue, payment capture, commission lines on issue	<code>policies</code> , <code>policy_payments</code> , <code>commissions</code>	Event <code>policy.issue.broker_billed</code> (through receipts); <code>policies.*</code> , <code>policy.*</code>

Module and routes (endpoints)	Purpose	Main tables	Jobs, posting events, settings
endorsements /endorsements (9)	Endorsements and their approval	endorsements	Events endorsement.additional_premium, endorsement.return_premium, policy.cancel; endorsements.types
cancellations /cancellations (3)	Computed cancellation: pro-rata, short-period, flat; return premium with taxes and commission taken back	reads policies, writes through endorsements	endorsements.cancellation_*, endorsements.short_period_for_insured
packages /packages (27)	Packaged products: bundles, insurer rate tables, quick quote comparison, package policies	package_bundles, package_quotes, package_sections, insurer_rate_tables	packages.*
premium-charges /premium-charges (9)	Premium tax engine: VAT or premium tax, DST, FST, LGT, other charges	premium_charge_rules, lgu_tax_rates	tax.charge_engine.default_lgu
product-configurator /product-configurator (90)	Product templates, components (covers, rating factors, acceptance rules, document templates, market mapping), risk mappings, governing template	product_templates, product_components, product_risk_mappings, product_risk_sections	product.*, underwriting.*
renewals /renewals, /policy-renewals (47)	Renewal pipeline, re-rating, notices, approval, batches, win-back	renewals, renewal_quotes, renewal_notices, renewal_batches	Jobs renewal-pipeline, renewal-notices, renewal-queue, policy-expiry; renewals.*
payment-gateway /payment-gateways, /payment-links, /public (13)	Payment gateways, payment links, public payment pages and webhooks	payment_gateways, payment_links, payment_events	payments.*; credentials in the environment
documents /document-templates (8)	Server-rendered PDFs of quotations, schedules and receipts	reads the record printed	documents.*, receipts.*

5.3 Distribution, motor and specialty lines

Module and routes (endpoints)	Purpose	Main tables	Jobs, posting events, settings
channels /channels (6)	Distribution channels: dealer groups and branches, financing banks, affinity partners	distribution_channels, channel_billing_accounts	channels.*
motor-programmes /motor-programmes (12)	Brand-new vehicle dealer programmes, dealer sales uploads, bank letters	motor_programmes, dealer_sales_batches, dealer_sales	motor_programmes.*
fleet /fleet (14)	Fleet schedules under one motor policy, vehicle additions and deletions by endorsement	fleet_schedules, fleet_vehicles	fleet.*

Module and routes (endpoints)	Purpose	Main tables	Jobs, posting events, settings
marine /marine (17)	Marine cargo open covers, certificates, monthly declarations	open_covers, open_cover_certificates, open_cover_declarations	marine.*
campaigns /campaigns (17)	Marketing campaigns to consenting clients and prospects, public opt-out link	campaigns, campaign_segments, campaign_templates, campaign_recipients	Job campaign-dispatch (off); campaigns.*
report-builder /report-builder (11)	Ad hoc reports over curated datasets, Excel export, BI extract	report_builder_reports, bi_extract_runs	Job bi-extract (off); report_builder.*, bi.*
sales-activities (package G) /sales-activities (8)	Sales activity log on prospects, quotations and clients; follow-ups in My Work	sales_activities, work_tasks	sales_activities.*

5.4 Claims

Module and routes (endpoints)	Purpose	Main tables	Jobs, posting events, settings
claims /claims (17)	Claim registration, lifecycle, field-level trail, settlement, Preliminary Loss Advice	claims, claim_history, claim_field_changes, claim_settlement_movements	Events claim.funds_received, claim.paid_to_claimant, claim.settlement.paid_through_broker; claims.*
claim-documents /claim-documents (6)	Claim document checklist, reminders, submission to the insurer	claim_document_items, claim_document_reminders	Job claim-document-reminders; claims.document_*
motor-claims /motor-claims (10)	Repair estimates, letters of authority, vehicle release	claim_repair_estimates, claim_loas, claim_vehicle_releases	motor_claims.*
claim-payments /claim-payments (6)	Claims Settlements in the Accounting menu: funds received and payment to the claimant	through claims and the receipt and voucher paths	Events claim.*

5.5 Accounting and finance

Module and routes (endpoints)	Purpose	Main tables	Jobs, posting events, settings
receipts /receipts, /billing-statement (19)	Billing (premium bills), official receipts, application to bills, billing statements	receivables, receipts, receipt_lines, receipt_applications, receivable_credits	Events policy.issue.broker_billed, receipt.apply; receipts.*, billing.*
collections /collections (9)	Collection follow-up of open bills, reminders	collection_items, collection_actions	Jobs receivable-ageing, collection-reminders; collections.*
credit-control /credit-control (20)	Instalment plans and invoices, premium warranty monitor, client credit limits	premium_instalment_plans, premium_instalments, premium_warranty_extensions, client_credit_exceptions	credit.*

Module and routes (endpoints)	Purpose	Main tables	Jobs, posting events, settings
		ons	
pdc /pdc (10)	Post-dated cheque register: deposit due list, deposit into a receipt, bounce, replacement	post_dated_cheques	Job pdc-deposit-due; pdc.*
disbursements /disbursements (15)	Payment vouchers, invoice lists, cheques	disbursements, invoice_lists, checkbooks	Event disbursement.payment; disbursements.*
payables /payables (14)	Accounts payable: supplier invoices with input VAT and EWT, payments, AP ageing	supplier_invoices, supplier_payments, supplier_payment_allocations	Events ap.invoice, ap.payment; payables.*
fixed-assets /fixed-assets (7; 13 with package G)	Fixed asset register, straight-line depreciation, disposal (package G)	fixed_assets, fixed_asset_depreciation, fixed_asset_disposals	Events fa.depreciation, fa.disposal; fixed_assets.*
payments /petty-cash, /open-items, /payments (29)	Petty cash funds and movements, open items, payment status	petty_cash_*, agent_events	Events pettycash.*
journal-vouchers /journal-vouchers (9)	Manual, reversal and correction journals with maker-checker	journal_vouchers, journal_lines	journal.require_approval
accounting /accounting (27)	Ledger queries, open-item matching, write-off, chart of accounts, periods	journal_vouchers, entry_matches, gl_accounts, accounting_periods	Events write_off, write_off.credit_balance; accounting.*
posting-rules /posting-rules, /account-determination (22)	Posting rules and Account Determination with maker-checker	posting_rules, posting_rule_lines, accounting_config_changes	accounting.configuration_maker_checker
period-end /period-end (45)	Fiscal calendar, month-end and year-end close, recurring journals, financial statements, tax codes, BIR Form 2307	period_close_runs, year_end_runs, recurring_journals, tax_codes, bir_2307_certificates	Jobs recurring-journals, accrual-reversal, period-auto-soft-close, month-end-reminder (off); accounting.*, period_end.*
bir /bir (39)	BIR returns 0619-E, 1601-EQ, 1604-E with DAT files, 2551Q, EOPT sales invoices, EIS outbox, CAS books pack	bir_return_filings, sales_invoices, eis_submissions, cas_book_prints	Job eis-outbox (off); events sales_invoice.issue, sales_invoice.payment; bir.*, invoice.*, eis.*, cas.*
bank-reconciliation /bank-reconciliation (38)	Bank accounts, statement import, matching, adjustments, monthly reconciliation	bank_statements, bank_rec_matches, bank_reconciliations	Job bank-auto-match (off); event bank.adjustment; bank_reconciliation.*
insurer-reconciliation /insurer-reconciliation (18)	Insurer statement import, matching, resolutions, approval	insurer_statements, insurer_statement_lines, insurer_statement_resolutions	Event insurer_statement.adjustment; insurer_reconciliation.*
remittance /remittance (86)	Remittances to insurers, approvals, direct bill, debit notes, refunds from insurers	remittances, remittance_lines, direct_bill_items, commission_debit_notes	Job remittance-schedules (off); events remittance.*, directbill.*, insurer.refund_*, remittance.*, direct_bill.*
commission /commission (19)	Referrer commission (comsub): accrual,	commissions, commission_adjustment	Events commission.approve,

Module and routes (endpoints)	Purpose	Main tables	Jobs, posting events, settings
	eligibility, approval, payout, clawback	s, commission_referrers	commission.payout, commission.clawback; commission.*
commission-rates /commission-rates (7)	Commission Rate Matrix with the overlap rule	commission_rates	commission.default_rate
insurer-overrides /insurer-overrides (15)	Overriding, profit and contingent commission from insurers	override_agreements, override_computations, override_settlements	Events override_commission.accrual, override_commission.settlement; commission.override_*
incentive /incentive (22)	Incentive programmes, calculations, approval, payout, statements	incentive_programs, incentive_calculations, incentive_results	Events incentive.accrual, incentive.payout; incentive.*
reinsurance /reinsurance (51)	Reinsurers, treaties, cessions, recoveries, bordereaux, reconciliation, facultative placement as reinsurance broker	reinsurance_treaties, cessions, reinsurance_recoveries, fac_placements, fac_settlements	Events ri.cession, ri.recovery, ri.facultative.*; reinsurance.*

5.6 Compliance

Module and routes (endpoints)	Purpose	Main tables	Jobs, posting events, settings
aml /aml (58)	AML/CFT: onboarding of juridical clients, beneficial owners, KYC documents, risk rating and EDD, screening lists and hits, covered and suspicious transaction monitoring, cases, AMLC report files	aml_*, client_signatories, client_beneficial_owners, client_kyc_documents	Jobs aml-transaction-monitoring, aml-kyc-refresh-due, aml-provider-retry (off); aml.*
privacy /privacy (14)	Consents, data subject requests, personal data export, anonymisation	privacy_consents, data_subject_requests	Job privacy-requests-due (off); privacy.*
ic-compliance (package B) /compliance (38)	Licence register and payout block, fit and proper, insurer authority check, IC annual statement and production report, complaints register (RA 11765)	compliance_licences, compliance_fit_proper, ic_statement_lines, complaints	Jobs compliance-reminders, complaints-deadlines; compliance.*, complaints.*
data-breaches (package B) /privacy/breaches (11)	Personal data breach register with the NPC 72-hour notification tracker	personal_data_breaches, personal_data_breach_reminders	Job privacy-breach-deadlines; privacy.breach_*

5.7 Integrations

Module and routes (endpoints)	Purpose	Main tables	Jobs, posting events, settings
integrations /integrations,	Integration framework (connectors, outbox,	integration_connectors, integration_outbox,	Jobs integration-outbox, sms-renewal-

Module and routes (endpoints)	Purpose	Main tables	Jobs, posting events, settings
/messaging, /ctpl, /insurer-integration, /bank-payments, /public/integrations (57)	inbox, monitor); SMS and Viber messaging; CTPL authentication and COC series; insurer API connectors; bank payment files	integration_inbox, coc_series, ctpl_authentications, bank_payment_batches	notices and sms-payment-reminders (off); integrations.*, messaging.*, ctpl.*, bank_payments.*

5.8 Platform, configuration and administration

Module and routes (endpoints)	Purpose	Main tables	Jobs, posting events, settings
auth /auth (15)	Sign-in, two-factor, refresh, password reset and change	users, refresh_tokens, password_resets	security.*, limits.max_login_attempts
users /users, /roles (17)	Users, roles and role permissions	users, roles, user_roles, role_permissions	
access-control /access-control (24)	Authority matrix, delegations, segregation of duties, access reviews	authority_limits, sod_rules, user_delegations, access_reviews	Job dormant-users; access.*
my-work /my-work (14)	My Items, My Team by reporting line, My Tasks, Calendar; reassignment	work_tasks	Job my-work-reminders
masters /masters (13)	Metadata-driven masters (generic and table-backed)	master_types, master_records	limits.bulk_upload_max_rows
ops-masters /ops-masters (4)	Operational masters kept by the teams that use them	master_records	
addresses /addresses (9)	Philippine address look-ups: region, province, city or municipality, barangay, ZIP code	reads the geography tables	
settings, system-settings /settings (6), /system-settings (7)	Configuration keys, system settings, audit trail endpoint of the settings screen	app_settings	system.*, branding.*
branding /branding (11)	Theme and Branding: themes, logo, sign-in picture, brand packs, branded documents and e-mails	app_settings, master_records	branding.*, documents.accent_color
e-signatures /e-signatures (7)	Signature capture, versions, revocation, mapping of signatures to document slots	e_signatures, document_signature_slots	signatures.*
document-numbering /document-numbering (5)	Document number series	document_numbering, sequences	
schedules /schedules (4)	Scheduled jobs: timing, enable, run now, history	scheduled_jobs, job_runs	
notifications /notifications, /email (11)	In-app notifications and the e-mail outbox	notifications, email_outbox	Job email-outbox; notification.*
reports /reports (13)	Report catalogue, runs, files, scheduled delivery	report_definitions, report_schedules, generated_reports	Job daily-reports; reports.*

Module and routes (endpoints)	Purpose	Main tables	Jobs, posting events, settings
dashboard /dashboard, /agent (6)	Dashboard figures	reads	dashboard.*
audit /audit (1)	Audit trail as business events (record timeline, Master > Audit Trail)	reads audit_log, claim_field_changes	
data-load /data-load (14)	Go-Live Data Workbench: configuration and migration kits, validation, errors workbook, load, reconciliation, environment comparison	data_load_batches, data_load_rows, data_load_comparisons	golive.*
uploads /s3, /upload (11)	File upload and download (/s3 API), signed links	documents	uploads.allowed_types
search /search (1)	Global search across leads, clients, quotations, policies, claims, endorsements	reads	
system /version (1)	Build and runtime information	none	

5.9 Scheduled jobs by module

35 jobs are seeded with package B (32 on the branch alone). 21 are enabled after a new installation (18 without package B); 14 are delivered switched off because they need a decision, a provider or a credential first.

Job	Module	Default timing (business time zone)	Delivered
email-outbox	notifications	every 5 minutes	on
integration-outbox	integrations	every 2 minutes	on
renewal-queue	renewals	every minute	on
my-work-reminders	my-work	every 15 minutes	on
policy-expiry, quote-expiry	policies, quotations	00:15, 00:30 daily	on
dormant-users, housekeeping	access-control, jobs	01:45, 02:45 daily	on
daily-reports, renewal-pipeline	reports, renewals	05:00, 05:30 daily	on
renewal-notice, cover-note-expiry, pdc-deposit-due, aml-transaction-monitoring, claim-document-reminders	renewals, cover-notes, pdc, aml, claim-documents	06:00, 06:20, 06:25, 06:30, 06:35 daily	on
receivable-ageing, collection-reminders	collections	07:00, 08:00 daily	on
aml-kyc-refresh-due	aml	07:00 every Monday	on
compliance-reminders, complaints-deadlines (package B)	ic-compliance	06:45, 08:00 daily	on
privacy-breach-deadlines (package B)	data-breaches	every hour at minute 15	on
recurring-journals, period-auto-soft-close,	period-end	01:15, 02:00 daily; 00:30 on the 1st; 08:00 daily	off

Job	Module	Default timing (business time zone)	Delivered
accrual-reversal, month-end-reminder			
bank-auto-match	bank-reconciliation	05:45 daily	off
remittance-schedules	remittance	06:15 daily	off
privacy-requests-due	privacy	07:00 daily	off
lead-assignment-sla	leads	07:30 daily	off
sms-renewal-notice, sms-payment-reminders	integrations	08:10, 08:20 daily	off
eis-outbox, campaign-dispatch, aml-provider-retry	bir, campaigns, aml	every 15 minutes	off
bi-extract	report-builder	02:00 daily	off

6. Platform engines

The modules share a small set of engines. A change to one of them reaches every module, so it is reviewed with the tests of all the modules that use it.

6.1 Route registry and API conventions

No module calls `router.get()` directly. `moduleRouter(label, prefix)` in `backend/src/lib/registry.js` returns `define()`, which places `requireAuth` first in the middleware chain unless the route is declared `auth: false`, wraps the handler so that rejected promises reach the error handler, and records the route with its summary, screen, roles, permissions and examples:

```
const define = (r) => {
  const method = r.method.toLowerCase();
  const auth = r.auth !== false;
  const given = (r.middleware || []).flat();
  const mws = auth ? [requireAuth, ...given.filter((m) => m !== requireAuth)] : given;
  router[method](r.path, ...mws, wrap(r.handler));
  ...
  ROUTES.push(entry);
};
```

A route therefore cannot be left open by forgetting `requireAuth`, and the API catalogue cannot drift from the code: `npm run export:api` writes OpenAPI 3.1, the Postman collection and the touchpoint workbook from the same registry, and `test/hardening.test.js` checks that every non-public route answers 401 without a token.

Topic	Convention
Base path	Every endpoint is under <code>/api</code> (for example <code>https://<host>/api/receipts</code>)
Authentication	Authorization: Bearer <access token> from <code>POST /api/auth/login</code> ; renewed with <code>POST /api/auth/refresh</code>
Success response	<code>{ "success": true, "message": "...", "data": ... }</code> ; 201 for created records
Lists	<code>page</code> and <code>perPage</code> (also <code>pageNo</code> , <code>pageSize</code> , <code>limit</code>); default 10 per page, at most 500; pagination: <code>{ total, page, perPage, totalPages }</code>
Errors	<code>{ "success": false, "message", "errors": [{ "path", "message" }], "requestId" }</code> ; 400 validation, 401 sign-in, 403 permission, 404 not found, 409 conflict, 413 too large, 415 file type, 429 rate limit, 500 server
Correlation	<code>x-request-id</code> header on every response; sent by the caller or created by the API
Files	PDF, XLSX and CSV downloads answer the file with <code>Content-Disposition</code> ; stored files are reached through signed links
Identifiers	Most detail routes accept the database id or the document number (<code>/receipts/OR-2026-00001</code>)
Dates and money	Dates as <code>YYYY-MM-DD</code> in the business time zone; amounts as numbers in PHP rounded to 2 decimals
Exports	List endpoints offer <code>export=csv</code> or <code>export=excel</code> where the screen has a download; the export is built during the request, so it is masked like the screen (package B)
Screen label	screen names the menu path (Accounts > Receipts > Add receipt); the help panel and the dependency workbook read it

6.2 Permissions and roles

- **Permission codes.** read:<area>, write:<area> and approve:<area> (98 codes with packages B and G), plus view:pii (package B: full personal identifiers). An endpoint names its permission with `requirePermission('write:receipts')`, or an administrator role with roles: [ADMIN_ROLE]. A missing permission answers 403 naming it.
- **Roles.** Eight roles are seeded. The System Administrator holds every permission. The grants of the other roles are in `ROLE_PERMS` of `backend/src/db/seed.js` and in the migration that added a permission; roles may inherit other roles (`ROLE_INHERITS`: Accounting Manager inherits Accounting; `user_effective_roles()` in the database). Administrators change grants on Master > User Management > Role Permissions.

Role (code)	Permissions	Typical grants
System Administrator (system-admin)	98	Every permission
Sales & Marketing (sales)	32	Leads, clients, quotations, policies, renewals, campaigns, dealer programmes, sales activities, view:pii
Processing Team (processing)	31	Quotations, policies, endorsements, products, reinsurance, fleet, marine, dealer programmes
Operations (operations)	40	Client servicing, endorsements, renewals, lead assignment, campaigns, privacy, complaints, compliance registers, view:pii
Claims (claims)	15	Claims, complaints, read of policies and clients
Accounting (accounting)	39	Receipts, collections, disbursements, payables, fixed assets, commission, remittance, journals, period end, bank reconciliation, view:pii
Accounting Manager (accounting-manager)	9 own, plus Accounting	The approvals: period end, bank and insurer reconciliation, credit control, payables, posting rules
Compliance Officer (compliance-officer)	15	AML/CFT (read:aml, write:aml, approve:aml), read of clients, policies, claims, receipts and payments

- **Record scoping.** Users whose roles are all listed in `security.scoped_roles` see only their own book (`scopeSql()`, `ownRecord()`).
- **Approvals.** Maker-checker (`lib/makerChecker.js`), the authority matrix (`assertAuthority()` in `access-control/service.js`) with delegations, and segregation-of-duties rules.
- **Front end.** `roleMenuPermissions` (deny by default) decides which menu entries a role sees; `hasPermission()` hides actions. The server stays the authority.

6.3 Posting rules engine

Every system journal is produced by `postEvent(eventCode, context)` in `backend/src/modules/accounting/lib/posting.js`. The active posting rule of the event (`posting_rules` and `posting_rule_lines`, the latest approved version effective on the posting date) is expanded into journal lines and handed to `createJournal`,

which validates balance, accounts and the period; the database trigger `jsv_check_posting` checks again.

- **Rule line:** side Dr or Cr; account as an account role (resolved through Account Determination, `accounting.account.<role>`), a fixed GL code, a resolver, or an account the operation supplies in `context.accounts`; the amount key (one of the amounts the event supplies); a per-participant flag (one line per co-insurer with its share); a narration template with `{{variables}}`. A negative amount posts on the opposite side; zero lines are dropped.
- **Events:** EVENTS in `posting.js` lists every business event with its label, module, amount keys, template variables and a sample context. 43 events have an active rule after seeding, from `policy.issue.broker_billed`, `receipt.apply` and `remittance.settlement` to `ap.invoice`, `fa.depreciation`, `sales_invoice.issue`, `override_commission accrual` and `ri.facultative.bind`. The Data Dictionary workbook (sheet Reference values) lists them.
- **Changes:** editing a rule creates a new version effective from a date. With `accounting.configuration_maker_checker` on, a change to a rule or to Account Determination is a pending row of `accounting_config_changes` that takes effect only when another user with `approve:posting-rules` approves it. Master > Finance > Posting Rules simulates a rule on a sample context before it is saved.
- **Adding an event:** add it to EVENTS with its amount keys and sample, seed its rule in a migration (lines with account roles, not fixed codes, so a broker's chart of accounts fits), call `postEvent()` inside the business transaction, and add a test that posts it and checks the journal.

6.4 Tax engine

Part	Where	What it does
Premium taxes and charges	<code>modules/premium-charges/calculator.js</code> , <code>chargesFor()</code>	One entry point for every premium (quick quote, bundles, quotations, endorsements, placement slips, insurer offers, renewals, cancellations). Rule kinds <code>vat</code> , <code>premium_tax</code> , <code>dst</code> , <code>fst</code> , <code>lgt</code> , <code>other</code> ; methods <code>percent</code> , <code>per_unit</code> , <code>flat</code> ; tax regimes <code>vat</code> , <code>premium_tax</code> , <code>exempt</code> ; rules in force on the date and per line of business; LGT from <code>lgu_tax_rates</code> per province or city (migration 0236 made it the single source)
Commission taxes	<code>modules/accounting/lib/commissionTax.js</code> , <code>tax_codes</code>	VAT on brokerage commission and expanded withholding tax by payee type, from tax codes with ATC (migration 0237)
Withholding certificates	<code>modules/period-end</code>	BIR Form 2307 certificates per payee and quarter; package G adds supplier certificates from payables
BIR returns and invoicing	<code>modules/bir</code>	0619-E, 1601-EQ, 1604-E with DAT files, 2551Q computed live from the ledger and the certificates; EOPT sales invoices; EIS submission; CAS books pack

Tax rates are configuration (Master > Finance > Premium Taxes & LGU Rates and Tax Codes); the keys owned by that screen are refused by the generic configuration endpoints (lib/settingOwners.js).

6.5 Document numbering

Every number the system issues comes from `nextDocumentNumber(code, { db, branch, lob, date, unique })` in `backend/src/lib/numbering.js`, over the series of the Document Numbering master (document_numbering, 85 series). The SQL function `next_document_number()` locks the counter row in sequences until the transaction ends, so two users never receive the same number and a cancelled save gives its number back. Pattern tokens: {PREFIX} {YYYY} {YY} {MM} {FY} {BRANCH} {LOB} {SEQ}; reset rules: yearly, fiscal_yearly, monthly, never. Migration 0246 lets a series start a period at the next number of the old system (go-live), and unique skips a number already present in the target column. A new document type is a new series seeded in its migration with `ON CONFLICT DO NOTHING`; a prefix must be unique among active series (index document_numbering_active_prefix).

6.6 Scheduled jobs and the scheduler

`backend/src/jobs/scheduler.js` reads `scheduled_jobs` and runs each enabled job with `node-cron` in the business time zone (`general.timezone`). Each run takes a PostgreSQL session advisory lock keyed on the job code, so a job runs once even when several API instances run the scheduler; `SCHEDULER_ENABLED=false` keeps an instance out of scheduling. A change to a schedule is picked up by the change watch without a restart. Every run is a row of `job_runs` with status, output and error. Handlers are exported by `backend/src/jobs/handlers.js`; a module keeps its own in `jobs.js` and the handlers file re-exports them. Administrators change the timing, switch a job on or off and run it at once on Master > System Configuration > Schedules. The 35 jobs are listed under "Scheduled jobs by module" in the module catalogue.

Rules for a job: the business date comes from `lib/dates.js` (never the database's `current_date`, which is the server's UTC date); the handler checks that its tables exist (the core runs before every module is migrated); it returns a JSON summary; it is idempotent for the same day; a job that needs a provider, a credential or a decision is seeded switched off.

6.7 PDF, Excel and e-mail engines with branding

Engine	Where	What it does
PDF	<code>lib/pdf/</code> (<code>renderPdf</code> , <code>renderPdfBatch</code> , <code>renderReportPdf</code> ; layout in <code>layout.js</code>)	The one PDF engine, without third-party libraries: documents (quotations, schedules, slips, receipts, vouchers, statements, claim letters, cover notes, LOAs, sales invoices) and report listings. Every page carries the letterhead and a footer with the company, "Generated <date time> by <user>" and "Page X of Y"; sections may carry signature blocks with images
Letterhead	<code>lib/letterhead.js</code>	The primary active company of the Company master (legal name, TIN,

Engine	Where	What it does
		registered address, print logo); the settings are only the fallback
Document branding	documentBranding() in modules/branding/service.js	Accent colour, logo and rule lines of documents and report files from the saved theme
Excel	lib/xlsx.js	Dependency-free writer: several sheets, styled and frozen header, autofilter, typed cells; report files carry the header colours of the theme and optionally the logo and a banner (company, registration line, title)
E-mail	lib/mailer.js, email_outbox, job email-outbox	Messages are queued in the business transaction and sent every 5 minutes through SMTP_URL when notification.email_enabled is on. At send time the body is wrapped in the broker's e-mail layout (emailLayout(): header band with the logo, footer line) and attachments are generated
Templates	lib/template.js	{{name}} placeholders in e-mail, notice and letter templates kept in settings; HTML-escaped for HTML bodies

Branding is data: Master > System Configuration > System Settings > Theme and Branding saves the theme (branding.theme, validated for WCAG AA contrast), the application logo, favicon and sign-in picture. GET /api/branding (public, with ETag) serves it; the front end's theme/runtime/themeEngine.js turns it into CSS custom properties on <html>, so screens, documents, reports and e-mails follow without a rebuild. Brand packs (exportBrandPack(), importBrandPack(), backend/scripts/build-brand-pack.js) move a whole branding between environments. A client brand pack such as the optional Toyota Insurance Services pack carries a third party's marks and is installed only with the client's written permission (docs/onboarding/BRANDING_AND_SIGNATURES.md).

6.8 E-signatures

modules/e-signatures keeps the signature images of authorised signatories (Master > Insurance Management > Signatories) and of users (My Profile): drawn or uploaded, with consent text, time and IP address, versions and revocation (e_signatures). document_signature_slots maps each document type and slot to the source of the signature (the signatory chosen on the document, a named signatory, the default signatory, the approving user or the issuing user) and a condition (only once issued or approved). The PDF engine prints the image only when the condition holds; drafts carry signatures.draft_watermark. Images are served only through the protected endpoint and are limited by signatures.max_bytes.

6.9 Integration framework

Every connection to a third party goes through one framework in backend/src/modules/integrations (migrations 0310 to 0314, reference data in seeds/75_integrations.sql). A new integration (the BIR EIS connector, a new SMS provider, an insurer with its own protocol) is an adapter and one or more message types registered in it; it then gets the settings, the outbox with retry, the inbox, the monitor screen and the audit trail without further work.

Part	File	What it does
Registry	framework/registry.js	registerAdapter({ code, label, kinds, credentialKeys, needsEndpoint, send, test }) and registerMessageType({ type, kind, label, onSent, onFailed, onRequeued, onInbound }); IntegrationError(message, { retryable, httpStatus, response })
Connectors	framework/connectors.js	integration_connectors: kind, adapter, enabled, mode test / live, endpoint, credential_env (environment variable NAMES per credential key), adapter options, timeout, attempts and backoff. Values are read from process.env at send time; the API returns only the names and whether each is set. Live mode is refused while the endpoint or a credential variable is missing (liveBlockers())
Outbox	framework/outbox.js	enqueue(db, {...}) inside the caller's transaction; processOutbox() picks due rows with FOR UPDATE SKIP LOCKED, sends, records integration_attempts, applies the answer with onSent under a savepoint, or schedules the next attempt after retry_base_seconds x 2^(attempt - 1) capped at retry_max_seconds; a non-retryable error or the last attempt marks the row failed and calls onFailed. Rows left processing longer than integrations.stuck_minutes are queued again
Inbox	framework/outbox.js	receive() stores and processes a pushed message or an imported file through the type's onInbound; POST /api/public/integrations/inbound/:connector verifies x-signature (HMAC-SHA256 of the raw body with the connector's webhookSecret credential)
Fake provider	adapters/fake.js	Every connector in test mode sends through it: deterministic answers, options.fakeFailFirst and options.fakeReject to exercise retry and failure
Live adapters	adapters/http.js	http_sms (presets semaphore, globe_labs, generic), viber_business, ctpl_http, lto_http, insurer_rest, file_drop (bank files). HTTP 429, 408, 5xx, timeouts and network errors are retryable; other 4xx are not
Hooks	hooks.js, messaging.js	afterPolicyIssued (CTPL registration and the insurer issuance request, under a savepoint so it never blocks the issue); claimStatusChanged

Business use	Tables	Message types
SMS and Viber messaging	message_templates	sms.send, viber.send
CTPL COC authentication	coc_series, ctpl_authentications	ctpl.authenticate, ctpl.lto_feed, inbound ctpl.authentication_result
Insurer API connectors	insurer_api_mappings	insurer.policy_issue, insurer.policy_data, insurer.claim_status, inbound insurer.claim_status, insurer.policy_issued
Bank payment files	bank_file_layouts, payee_bank_accounts, bank_payment_batches, bank_payment_batch_lines	bank.payment_file, inbound bank.status_file

Every connector is delivered in test mode. Going live with a provider needs the provider's credentials in the environment and its certification with the broker; the roadmap in the Release Notes and Roadmap lists the partner certifications still to be done.

6.10 Encryption and masking of personal data

Control	Where	What it does
Passwords, reset codes	lib/password.js, modules/auth	bcrypt hashes (cost 10); reset codes as keyed hashes
Two-factor secrets	lib/secrets.js	AES-256-GCM with a key derived from DATA_ENCRYPTION_KEY
Personal identifiers at rest (package B, migration 0277)	lib/pii.js, database triggers pii_protect_columns and pii_protect_json	TIN, government ID and bank account numbers stored as pii:1:<key id>:<iv + AES-256-CBC ciphertext>:<HMAC-SHA256 tag> in clients.tin, leads.tax_number, commission_referrers.tin and bank_account_no, bir_2307_certificates.payee_tin and the identifier keys of JSON documents and audit values. Blind indexes *_bidx (keyed HMAC of the normalised value) keep exact search. The keys are derived from PII_ENCRYPTION_KEY and reach the database as session settings of every pool connection, so every writer (screens, seeds, go-live loaders) stores ciphertext; a session without them cannot write an identifier
Masking by role (package B, migration 0276)	lib/piiPolicy.js, piiMiddleware	Users without view:pii receive TIN, ID, mobile, e-mail, bank account and birth date partially masked in every GET answer and in the exports built during the request (Excel, CSV, report PDF). Settings: privacy.masking_enabled, privacy.pii_reveal_mode (always, or on-request: holders switch on "Show full identifiers" in the user menu, the header X-Unmask-PII: 1 is sent and each unmasked answer is audited as entity personal_data, action unmask), privacy.masking_exempt_paths
Personal data catalogue	backend/scripts/lib/pii-catalogue.js	One list drives the masking by role, the JSON keys encrypted and the client data masking tool

Key rotation (deploy/REFERENCE.md with package B): generate a new key (openssl rand -hex 32); set PII_ENCRYPTION_KEY to it and PII_ENCRYPTION_KEY_PREVIOUS to the old key and restart (both are readable, new values use the new key); run npm run pii:rotate (dry run counting the values still on the old key), then npm run pii:rotate -- --execute (batches, safe to repeat); when every count is 0, remove PII_ENCRYPTION_KEY_PREVIOUS and restart. Keep the old key with the backups taken before the rotation. In production the API refuses to start when PII_ENCRYPTION_KEY is

missing, a placeholder, shorter than 32 characters or equal to JWT_SECRET or DATA_ENCRYPTION_KEY.

6.11 Audit trail

Every handler that changes data calls `audit(req, { entity, entityId, action, before, after })` after the change. `audit_log` records the user, IP address, time, before and after images and, since migration 0247, the source (screen with the route's screen label, `api` for a call from outside the browser, `job` for a scheduled job). `lib/auditEvents.js` reads the rows back as business events: one per action, with each changed field as label, old value and new value formatted as on screen, secrets never shown, ID and bank numbers masked without `read:privacy`, anonymised parties shown as "Anonymised". The events are served by GET `/api/audit/records/:entity/:id` (the History panel of a record, for users who may read its module) and GET `/api/settings/audit/events` (Master > System Configuration > Audit Trail, with filters and Excel or CSV export, administrators). Domain histories add detail: `claim_history`, `claim_field_changes`, `period_status_history`, `bank_reconciliation_history`, `login_history`, `job_runs`.

6.12 Go-live workbench and data tools

Tool	Where	What it does
Go-Live Data Workbench	Master > Go-Live and Data > Go-Live Data Load; <code>modules/data-load</code>	Configuration kit (company, settings, organisation, users, finance and insurance masters, commission rates, taxes, authority limits, numbering) and migration kit (clients, in-force policies, open bills, open claims, opening balances). Validation is a dry run of every sheet in load order inside one ambient transaction rolled back at the end, each row under its own savepoint; the errors workbook lists every failing row; the load commits only when no row fails; the reconciliation compares loaded counts and totals with the workbook (<code>data_load_batches</code> , <code>data_load_rows</code>)
Environment comparison	Workbench > Compare environments; <code>backend/scripts/compare-environments.js</code>	Compares the configuration and masters of two environments on the natural keys of the workbench and writes the comparison workbook with the verdict (<code>data_load_comparisons</code>)
Transaction reset	<code>npm run reset:transactions</code>	Removes every business record of a smoke test and keeps masters, configuration and users; refuses while <code>golive.locked</code> is on or while a table is missing from <code>backend/scripts/lib/table-classification.js</code>
Client data masking	<code>npm run mask:data</code>	Masks personal data in a copy restored from production (<code>--environment</code> , <code>--execute</code> , <code>--verify-only</code> , <code>--remark-copy</code> to re-mark a copy still marked production, <code>--register-production</code> run once in Production); refuses on a database marked production (<code>system.environment</code>); marks the copy with <code>system.masked_at</code>
Go-live	<code>npm run rehearsal:golive</code>	Runs the configuration and migration

Tool	Where	What it does
rehearsal		loads end to end on a rehearsal database
Sample data purge	npm run purge:sample	Removes the demo masters and users of the sample seed

6.13 Release pipeline

The pipeline is described in `deploy/RELEASE_PIPELINES.md`. In short: one build per commit (artefacts `web-<sha>` and `backend-<sha>`, kept 90 days) promoted unchanged through Dev, SIT (large brokers), UAT, a temporary Pre-Prod restored from a production backup, and Production.

Ref	Deploys to	Approval
brokerverse-platform (trunk)	dev, automatically	none
vX.Y.Z-rc.N tag	uat automatically, sit by hand	iorta TechNXT delivery lead
vX.Y.Z tag	production automatically, preprod by hand before it	two reviewers (DevOps lead and the broker's release approver or CAB)

`deploy.yml` checks the environment is configured, waits for approval, takes a verified pre-deploy backup (always for preprod and production), installs the release beside the running one, migrates (forward only) and seeds, switches PM2, publishes the front end with its runtime `env-config.js`, runs `deploy/smoke-test.sh` and rolls back by itself when the switch or the smoke test fails. `rollback.yml` switches an environment back to an earlier build without touching the database. Migrations are forward only and expand-then-contract, so the previous release always runs on the newer schema.

7. Coding standards followed in this code base

This chapter records the conventions the existing code follows, with real excerpts. New code is expected to follow the same rules; the chapter "How to review a change" turns them into the reviewer's checklist.

7.1 Naming and file layout

- Back end: one folder per module in kebab case (bank-reconciliation, insurer-overrides); files in camel case (bundleQuotes.js, periodEndQueries.js); functions in camel case; constants in upper case (HEADER_FIELDS, PAYMENT_MODES).
- Database: tables and columns in snake case (receipt_lines, created_by); API fields in camel case (receiptNumber). Services map between them (for example HEADER_FIELDS in receipts/service.js).
- Permissions are read:<area>, write:<area> and approve:<area> (98 codes). Settings are <group>.<name> (limits.quote_validity_days), with a label and a group so that Master > Configuration can show them.
- Front end: React components in Pascal case under module/<Area>/ or agentModule/<area>/; one service file per API area named <area>Service.js; translation keys grouped by screen (myWork.calendar, aml.*).

7.2 Validation with zod

Request bodies and query strings are validated with zod schemas through `validate(schema, part)` (backend/src/lib/validate.js). The parsed value replaces the original, so the handler receives typed data. A failure answers 400 with "Validation failed" and one message per field ("amount is required").

```
export const validate = (schema, part = 'body') => (req, _res, next) => {
  const r = schema.safeParse(req[part]);
  if (!r.success) return next(r.error);
  req[part] = r.data;
  return next();
};
```

Business checks that need the database (an open period, an existing policy, the approver's authority, a referrer's licence) are made in the service and throw the errors below.

7.3 Errors and the response envelope

Services throw the helpers of backend/src/lib/errors.js (badRequest, forbidden, notFound, conflict, or `HttpError` with any status). `errorHandler` turns every failure into one envelope:

```
{ "success": false, "message": "Validation failed",
  "errors": [{ "path": "amount", "message": "amount is required" }],
  "requestId": "6d1c..." }
```

A server error (5xx) never returns its detail to the browser. The body reads "Internal server error; quote the request id when reporting it" and the message, stack and

database error go to the log with the same request id. Successful answers use `ok()` and `created()` from `backend/src/lib/respond.js`: { success: true, message, data }, with pagination on lists.

7.4 SQL: parameters, pool and transactions

All SQL is plain text in the services and goes through the pg pool with numbered parameters. Values are never pasted into SQL text. An update of a receipt header (`backend/src/modules/receipts/service.js`) shows the usual pattern for partial updates: the column names come from a fixed map in the code, the values from parameters.

```
const sets = []; const p = [r.id];
for (const [k, col] of Object.entries(HEADER_FIELDS))
  if (b[k] !== undefined && b[k] !== null) { p.push(b[k]); sets.push(`${col} = $$
{p.length}`); }
p.push(user.id); sets.push(`updated_by = $$${p.length}`, 'updated_at = now()');
await db.query(`UPDATE receipts SET ${sets.join(', ')} WHERE id = $1`, p);
```

Operations that write more than one table run in `withTransaction()` (`backend/src/db/pool.js`): `BEGIN`, the function, `COMMIT`, or `ROLLBACK` on any error. Services take the client (`db`) as their first argument so that callers can compose them in one transaction; numbering, posting, the integration outbox and the e-mail outbox all join the caller's transaction. Rows that are changed under concurrency are read with `FOR UPDATE` first. The section "SQL injection prevention" lists how the few dynamic identifiers are controlled.

7.5 Configuration through app_settings

Business parameters are not written into the code. They are rows of `app_settings` (660 keys in 64 groups after seeding), read with `getSetting(key, fallback)` and edited on Master > Configuration and Master > System Settings, or on the screen that owns them (`lib/settingOwners.js`). For example (`backend/src/modules/quotations/service.js`):

```
const validity = Number(await getSetting('limits.quote_validity_days', 30));
```

Settings are cached per API instance; the cache is checked against the table at most every 5 seconds (`SETTINGS_CHECK_MS`) and every entry expires after 60 seconds (`SETTINGS_CACHE_TTL_MS`), so a change saved on one instance reaches the others without a restart. A new key is added in a seed or migration with `ON CONFLICT (key) DO NOTHING` and a label; `npm run check:settings` fails when the code reads a key that no migration or seed creates.

7.6 Secrets and environment

Deployment values and secrets come from the environment only (`backend/src/config.js`, `deploy/backend.env.example`): `DATABASE_URL`, `JWT_SECRET`, `DATA_ENCRYPTION_KEY`, `PII_ENCRYPTION_KEY` (package B), `CORS_ORIGINS`, `PUBLIC_BASE_URL`, `SMTP_URL`, `APP_ENVIRONMENT`, upload limits and token lifetimes. Payment gateway credentials are read from `<prefix>_SECRET_KEY` and `<prefix>_WEBHOOK_SECRET` variables; integration connectors store only the names of their credential variables (`credential_env`). The development fallbacks in `config.js`

are refused in production: with `NODE_ENV=production` the server does not start when `JWT_SECRET`, `DATA_ENCRYPTION_KEY` or `PII_ENCRYPTION_KEY` is missing, a placeholder, shorter than 32 characters or equal to another of them, when `CORS_ORIGINS` is `*`, or when `PUBLIC_BASE_URL` is missing or a localhost address.

7.7 Logging

The back end logs JSON lines to standard output with pino (`backend/src/lib/logger.js`). `console.log` is an ESLint error in application code. Each request line carries the request id; job lines carry the job code and run id. Authorization headers, cookies, passwords, refresh tokens, one-time codes and signed-link parameters are redacted (`REDACT_PATHS` and `redactUrl`). The log level comes from `LOG_LEVEL`. In the front end, `utility/logger.js` replaces direct `console` calls (`no-console` is a warning there).

7.8 Front-end conventions

- Every HTTP call goes through a file in `services/`; screens do not build URLs themselves.
- Screen texts come from `locales/en.json` through `useTranslation()` and `t("key")`; `npm run check:il8n` lists keys used in code and missing from `en.json`.
- Menu visibility and route access come from `roleMenuPermissions` (deny by default). Buttons for actions the API would refuse are hidden with `hasPermission("write:...")`; the server stays the authority.
- Each screen has a help entry: `components/HelpPanel/helpRoutes.js` maps its address to a heading of the user manual; `npm run help:build` rebuilds `public/help/` from `docs/package/source/user-manual.md` and `helpRoutes.test.js` checks every heading id.
- Colours come from the theme tokens (`theme/bdoi/tokens.scss`) as CSS custom properties set at run time by the theme engine; a literal brand colour in a stylesheet is mapped to a token by `scripts/postcss-brand-vars.js`. Status colours (success, warning, danger) are not themed.
- Lists use `components/DataTable` (skeleton rows while loading, numeric columns right-aligned); add and edit forms open as side panels and return to the list after saving.

8. Code quality controls

8.1 Size of the code base

Area	Files	Lines
Back end: modules (src/modules)	281	55,985
Back end: shared libraries (src/lib)	44	3,856
Back end: migrations	139	8,360
Back end: seeds (reference and sample)	46	9,678
Back end: jobs	3	366
Back end: tests (test/)	99	20,021
Back end: scripts	51	7,595
Front end: module/	465	108,822
Front end: agentModule/	244	57,182
Front end: services/	59	11,752
Front end: components	38	4,922
Front end: theme and styles	7	7,958

Inventory item	Count
Back-end module folders	73 (70 before packages B and G)
Endpoints in the route registry (plus 2 health endpoints)	1,356 (1,291 before packages B and G)
Menu screens / route elements in MainRoute.js	223 / 473
Front-end service files / API call sites found by the checker	57 / 1,190
Database tables / views / functions (with packages B and G)	260 / 2 / 73
Settings / permissions / roles / master types / report definitions / number series / jobs	660 / 98 / 8 / 69 / 40 / 85 / 35

8.2 Static checks

- **Back end:** ESLint 9 with backend/eslint.config.js: the recommended rule set plus no-console (error), no-unused-vars (error, arguments starting with _ allowed), eqeqeq (smart) and prefer-const. Command-line scripts may write to the console. npm run lint on 04 October 2026: no errors, no warnings.
- **Front end:** npm run lint (eslint --ext .js,.jsx src) with the Create React App rule set (react-app, react-app/jest) and no-console as a warning. Run on 04 October 2026: 961 files, no errors, 256 warnings (134 react-hooks/exhaustive-deps, 105 eqeqeq, 17 others). Warnings do not stop the build; new code should not add any.

8.3 Automated tests

- **Back end:** vitest, 104 test files with 1,113 tests, run against a real PostgreSQL database (TEST_DATABASE_URL, default brokerverse_test) with fileParallelism: false. The suite covers every business module through the HTTP API with supertest (for example receipts.test.js, aml.test.js, bir-forms.test.js, integrations.test.js, my-work.test.js), the security controls (security.test.js, hardening.test.js, role-access.test.js, scope.test.js), configuration (configuration.test.js, settings-ownership.test.js), the data tools (go-live-workbench.test.js, environment-comparison.test.js, reset-

transactions.test.js, mask-data.test.js) and the printing and branding (printing.test.js, branding.test.js). Result on 04 October 2026 on the merged release: all 1,113 tests passed in one run of about 11 minutes.

- **Front end:** jest through craco test, 33 suites with 175 tests: menu tree and route permissions (utils/menuPermissions.test.js, components/SideBar/menuTree.test.js), help routes, the theme engine, the Philippine address fields, the Product Configurator rules, My Work, BIR Tax, Compliance and IC compliance, Distribution, Sales Activities and Integrations screens, number and date formatting. Result on 04 October 2026: all 175 tests passed.

8.4 Consistency checks

Command	What it checks	Result on 04 October 2026
npm run check:settings (back end, DATABASE_URL of a seeded database)	Every key read in code exists in a migration or seed; lists unread keys	0 missing; 9 read only by the front end; the keys not read by the branch code are mostly those of packages B and G (applied to the database, code not yet merged)
npm run check:api (front end)	Every front-end call matches a back-end route and method	1,190 call sites, 0 calls to missing routes; 2 method warnings (packagesService.js posts to two PDF routes that accept GET); 89 routes no screen calls
npm run check:i18n (front end)	Translation keys used in code and missing from en.json	25 keys without an English text (audit timeline, claim audit trail, currency master, policy history); 4,084 keys not yet in th.json
npm run export:api (back end)	Loads every module and writes OpenAPI, Postman and the touchpoint workbook	1,291 routes, no module skipped

check:api compares against backend/docs/api/openapi.json. Run npm run export:api in backend/ first, so that the check uses the current routes.

8.5 Migration discipline

- One new file per change, numbered after the last one in the range given to the change; applied files are never edited.
- Each file runs in its own transaction; a failure stops the start-up with the file name in the log and leaves the database at the previous file.
- Migrations only add or convert: new tables and columns, IF NOT EXISTS, ON CONFLICT DO NOTHING, updates with a WHERE clause. A column or table is removed only in a later release, once no deployed release reads it.
- The runner takes a PostgreSQL advisory lock, so several API instances starting together apply the files once.
- GET /api/health answers 503 while migrations are pending; GET /api/version lists them.

8.6 UAT scenario script

backend/scripts/uat-scenario.js runs the full broking cycle of a Philippine non-life broker through the public HTTP API, with synthetic data spread over six months: personas, masters, sales, placement, billing, servicing, money, reconciliation, month-end and reports. The phases are in backend/scripts/uat/phases/. It is pointed at any environment with API_BASE, ADMIN_PASSWORD and PERSONA_PASSWORD, uses a fixed random seed so that a rerun gives the same data, and exits with code 1 when a step fails.

8.7 Continuous integration

.github/workflows/ci.yml runs on every pull request and push:

Job (required status check)	Steps
Backend lint and tests	npm ci, npm run lint, npm test (the full suite against a postgres:16 service)
Front-end lint, tests and build	npm ci --legacy-peer-deps, npm run lint, craco test, environment-neutral build, checks on the build output; artefact web-<sha>
Backend release artefact	deploy/package-backend.sh; artefact backend-<sha>
Dependency audit	npm audit --omit=dev --audit-level=high for back end and front end

Branch protection on brokerverse-platform requires a pull request, one approving review and the four checks. After they pass, CI calls deploy.yml for dev (trunk), uat (vX.Y.Z-rc.N) or production (vX.Y.Z).

9. Security controls in code

9.1 Authentication

- **Tokens.** Sign-in (POST /api/auth/login) returns a short access token (JWT, HS256 only, JWT_ACCESS_TTL_SECONDS, default 1,800 seconds) and a refresh token (default 30 days). Refresh tokens are stored in refresh_tokens by id, rotate on every use, and a reused refresh token revokes its whole family.
- **Every request** is checked by authenticate() in backend/src/lib/auth.js: signature and algorithm, token type access, the user still exists and is active, and the token version matches users.token_version. Raising the token version (password change or reset, deactivation, role change, Sign out everywhere) ends every session of that user at once.
- **Restricted sessions.** A user who must change the password or enrol in two-factor sign-in receives a restricted token that only reaches the change-password or enrolment endpoints.
- **Passwords** are hashed with bcrypt (cost 10). An unknown user name still costs one bcrypt comparison, so response time does not reveal which names exist. Password reset codes are stored as keyed hashes, expire after security.reset_code_minutes (15) and are withdrawn after security.reset_code_max_attempts (5) wrong entries.
- **Idle sign-out** in the browser after limits.session_idle_minutes (default 30) with a one-minute warning (brokerverse/src/utility/idleTimeout.js).

9.2 Password policy, lockout and two-factor sign-in

Control	Setting	Seeded value
Minimum length	security.password_min_length	8
Upper case, lower case, digit, symbol	security.password_require_*	all required
Password history	security.password_history_count	last 5 refused
Maximum age	security.password_max_age_days	90 days
Lockout after wrong passwords	limits.max_login_attempts	5 (status becomes locked)
Sign-in attempts per IP and per user name	security.login_rate_limit	10 per 300 seconds
Roles that must use two-factor sign-in	security.require_2fa_roles	none

Two-factor sign-in uses authenticator codes (TOTP, RFC 6238, backend/src/lib/totp.js). The secret is stored encrypted, a code cannot be used twice (totp_last_step), and wrong codes count against the sign-in rate limit. Administrators reset a user's two-factor enrolment with POST /api/users/:id/2fa/reset.

9.3 Authorisation

- **Permission on every route.** Of the 1,291 endpoints, 1,200 require a permission or an administrator role through requirePermission() or roles, 67 require only a signed-in user (own profile, own notifications and tasks, address look-ups, file upload and download, global search, record history checked in

the handler) and 24 are public by design (see "Public endpoints"). A missing permission answers 403 naming the permission.

- **Roles, scoping and approvals.** See "Permissions and roles" in the chapter Platform engines.
- **Compliance controls in the flow.** AML/CFT onboarding blocks the first policy of a juridical client until its due diligence is complete (`aml.block_issue_pending_edd`); with package B the licence register blocks payouts to referrers whose licence has lapsed and the insurer authority check warns or blocks per `compliance.insurer_authority_check`.
- **Front end.** `roleMenuPermissions` denies by default; `ProtectedRoute` applies the same rule to typed URLs. This only hides screens; the server enforces the permissions.

9.4 SQL injection prevention

Method. Every JavaScript file under `backend/src` is searched for (a) SQL text built with `+` concatenation, (b) template literals that contain SQL keywords and interpolate a value (`${...}`), and (c) interpolations that reference `req.query`, `req.body` or `req.params` directly. Each interpolated expression found in (b) is traced to its source. The method was applied in full on 03 October 2026 and to the modules added since then on 04 October 2026.

Result.

- No SQL is built by string concatenation.
- No request value is interpolated into SQL text. The hits for (c) are messages and audit action names, not SQL.
- Every interpolated expression is one of: a placeholder number (`$$ {params.length + 1}`), a WHERE clause assembled from fixed fragments whose values are parameters, a constant SQL fragment defined in the same file, or an identifier chosen from a fixed list in the code. All values go to `pg` as parameters.

The dynamic identifiers and how each is controlled:

Where	What is interpolated	Control
Sorting in lists (<code>renewals/batches.js</code> , <code>collections/service.js</code> , <code>claims/service.js</code> , <code>my-work</code>)	ORDER BY column and direction	Column looked up in a fixed map; direction is ASC or DESC only
Partial updates (receipts, disbursements, claims, renewals, document numbering, product configurator, payables, fixed assets)	Column names in SET	Taken from fixed field maps in the service, never from the request
Generic inserts (clients, leads, quotations, placements, package quotes)	Column list	Keys of an object built in code from a fixed field map
Masters (<code>masters/service.js</code>)	Table and column names of table-backed masters	Table must be in the TABLES allow-list; identifiers pass IDENT (<code>^[a-z_][a-z0-9_]*\$</code>) and are double-quoted by <code>q()</code>
Report engine (<code>reports/engine.js</code>) and	Report query, columns, group-by, filters	Queries and datasets come from code; column keys pass

Where	What is interpolated	Control
Report Builder (report-builder/datasets.js)		IDENT or are looked up in the dataset definition; filters come from fixed maps
Go-live workbench (data-load)	Target table and columns of a sheet	Sheet definitions in code; workbook headers are matched against them
Petty cash, numbering, housekeeping, PII rotation	Table and column names	Fixed maps or lists in the code; rotate-pii-key.js quotes identifiers from PII_STORAGE

9.5 Input validation and request limits

- Bodies and query strings are validated with zod; services check business rules against the database.
- JSON bodies are limited to JSON_BODY_LIMIT (2mb). Uploads are held in memory and limited to UPLOAD_MAX_MB (10 MB) per file and UPLOAD_MAX_FILES (10) per request; imports to one file of IMPORT_MAX_MB (10 MB), IMPORT_MAX_INFLATED_MB (50 MB) once uncompressed, and IMPORT_MAX_ROWS (20,000) rows. Module limits apply on top (fleet.max_upload_rows, motor_programmes.max_rows, report_builder.max_rows).
- Lists cap the page size at 500 rows (paging() in respond.js); report files cap at 50,000 rows.

9.6 HTTP headers, CORS and rate limiting

- helmet sets the security headers on every response; x-powered-by is off. The web server adds its Content-Security-Policy (script-src 'self'), which is why the runtime configuration is a separate /env-config.js file.
- CORS allows only the origins in CORS_ORIGINS (* is refused in production).
- The global limiter allows security.api_rate_limit requests per window (600 per 60 seconds) per signed-in user, or per IP without a valid token; the answer is 429 with Retry-After. Both limiters count per API instance.
- trust proxy is set to one hop, so the client address is taken from the load balancer's header.

9.7 File uploads and downloads

- The type of an uploaded file is decided from its content signature and extension, never from the type the browser sends (backend/src/modules/uploads/fileTypes.js). Only types in uploads.allowed_types are accepted: JPEG, PNG, GIF, WebP, PDF, CSV, plain text, Excel and Word. Branding images and signatures have their own size and type limits (uploads.image_*, signatures.max_bytes).
- Stored files get a key with a 128-bit random part (storage.js), so keys cannot be guessed.
- Downloads need a bearer token or a signed link (?exp=&sig=, HMAC, valid FILE_URL_TTL_SECONDS, default 1,800 seconds). Files are served with X-Content-Type-Options: nosniff; everything except PDF gets a sandbox

content security policy; only images and PDF open inline; HTML is never served as HTML.

- Replacing or deleting a file is limited to the uploader, a user with write permission on the module that owns it, or an administrator, and is audited.

9.8 Encryption and secrets

- Two-factor secrets: AES-256-GCM with a key derived from `DATA_ENCRYPTION_KEY`. One-time codes are kept as HMAC hashes and compared in constant time.
- Personal identifiers (package B): see "Encryption and masking of personal data" in the chapter Platform engines.
- `DATA_ENCRYPTION_KEY` and `PII_ENCRYPTION_KEY` must be kept with the database backups: without the first the stored two-factor secrets cannot be read and users must enrol again; without the second TIN, ID and bank account numbers cannot be read.
- `JWT_SECRET` signs tokens and, through a derived key, file links. Changing it ends every session and every issued link.
- No secret is in the repository: `.env` files are local; examples carry placeholders that production refuses.

9.9 Audit and sign-in logging

- `audit_log`: every change with user, IP address, source, entity, action and before and after images; read as business events (see "Audit trail" in the chapter Platform engines).
- `login_history`: every sign-in attempt with result and reason (bad-password, unknown-user, locked, inactive, rate-limited, 2fa-required, bad-2fa-code).
- `job_runs`: every scheduled or manual job run with status, output and error; `integration_attempts`: every call to a third party.
- Request logs with request id and redaction.

9.10 OWASP Top 10 (2021) mapping

OWASP risk	Controls in BrokerVerse	Where in the code
A01 Broken access control	Permission on every route; deny-by-default menu; record scoping; maker-checker; authority matrix; masking by role	<code>lib/auth.js</code> , <code>lib/registry.js</code> , <code>lib/scope.js</code> , <code>lib/makerChecker.js</code> , <code>lib/piiPolicy.js</code> , <code>utils/menuPermissions.js</code>
A02 Cryptographic failures	<code>bcrypt</code> passwords; AES-256-GCM for two-factor secrets; AES-256-CBC with HMAC for personal identifiers; HMAC codes and links; HS256 only	<code>lib/password.js</code> , <code>lib/secrets.js</code> , <code>lib/pii.js</code> , <code>lib/auth.js</code>
A03 Injection	Parameterised SQL only; identifiers from allow-lists; zod validation	All services
A04 Insecure design	Restricted sessions; token versions; refresh rotation with reuse detection; segregation of duties; go-live lock;	<code>lib/auth.js</code> , <code>modules/access-control</code> , <code>lib/goLiveLock.js</code> , <code>lib/environment.js</code>

OWASP risk	Controls in BrokerVerse	Where in the code
	production marker refused by the masking tool	
A05 Security misconfiguration	Production start-up check; helmet; CORS list; no stack traces to clients; settings owned by one screen	config.js, app.js, lib/errors.js, lib/settingOwners.js
A06 Vulnerable and outdated components	Locked dependency versions; npm ci; npm audit gate in CI	package-lock.json, .github/workflows/ci.yml
A07 Identification and authentication failures	Password policy and history; lockout; sign-in rate limit; two-factor sign-in; idle sign-out	lib/password.js, lib/rateLimit.js, lib/totp.js, utility/idleTimeout.js
A08 Software and data integrity failures	Signed payment and integration webhooks checked against the raw body; migrations in transactions; seeds idempotent; one artefact promoted unchanged	modules/payment-gateway/providers.js, modules/integrations/framework, db/migrate.js, deploy.yml
A09 Security logging and monitoring failures	Audit log with source; sign-in history; request ids; log redaction; job runs; integration attempts	lib/audit.js, lib/loginHistory.js, lib/logger.js
A10 Server-side request forgery	The API calls only configured addresses (payment gateways, SMTP, integration connector endpoints set by an administrator); no request-supplied URL is fetched	modules/payment-gateway/providers.js, modules/integrations/adapters, lib/mailer.js

9.11 Points for the security review before go-live

These are facts of the current design that the customer's security reviewer should confirm as acceptable.

- GET /api/settings returns every configuration value to any signed-in user. No secret is stored in settings, but security parameters (lockout, rate limits) are visible to all staff.
- A stored file can be read by any signed-in user who has its key. Keys are random and are only handed out in API responses to users who can see the record.
- Access and refresh tokens are kept in the browser's local storage.
- Rate-limit counters are kept in memory per API instance; with several instances behind a load balancer the effective limit is multiplied by the number of instances.
- security.require_2fa_roles is empty after seeding; the customer decides which roles must use two-factor sign-in.
- Integration connector endpoints are entered by administrators; the reviewer should confirm who holds write:integrations and which outbound hosts the network allows.
- TIN, ID and bank account numbers are encrypted at rest since package B (migration 0277; 0331 adds the government ID numbers of the AML/CFT onboarding); PII_ENCRYPTION_KEY must be set before the migrations run on an installation, or the identifiers cannot be stored.

10. Front end to back end dependencies

10.1 How the matrix was built

The matrix answers, for each menu screen, which front-end file, which service files and which endpoints it uses, which back-end module serves them, and which data must exist before the screen can be used. It was built from the code by `docs/package/tools/build_api_catalogue.py`:

1. The 223 menu screens were read from `brokerverse/src/components/SideBar/list.js` (path and the includes list of related pages).
2. Each path was resolved to its component through `brokerverse/src/routes/MainRoute.js`, and the component to its file.
3. The import closure of each component gave the service files and the service functions the screen uses.
4. The API call sites in those files were taken from the parser in `brokerverse/scripts/check-api-calls.js` and matched to the route registry. Endpoints whose registry screen label names the menu item were added.
5. The back-end module of each endpoint and the module prerequisites complete the row.

The complete result, one row per menu screen, is the sheet **Screen dependencies** of the companion workbook.

10.2 Summary per menu area

Menu area	Screens	Main service files	Back-end modules
Home	1	clientService, dashboardService, paymentsService, policyService	clients, dashboard, payments, policies
Dashboard	4	dashboardService, reportsService, clientService, policyService	dashboard, reports, claims, clients, policies
Operations > Sales & Marketing	10	placementService, distributionService, mastersService, quotationService	placement, leads, clients, masters, premium-charges
Operations	12	opsAccountingService, clientService, mastersService, claimsService	masters, claims, clients, endorsements, documents
Operations > Renewals	8	renewalsWorkspaceService, batchRenewalService	renewals, settings
Accounts	13	accountingService, mastersService, journalVoucherService, clientService	accounting, masters, journal-vouchers, clients, disbursements
Accounts > Credit Control	4	creditControlService, remittanceService	credit-control, masters
Accounts > Payables	4	opsAccountingService	ops-masters, payables, masters, fixed-assets
Accounts > Fixed Assets	2	opsAccountingService	fixed-assets
Accounts >	16	remittanceService,	remittance, masters, settings,

Menu area	Screens	Main service files	Back-end modules
Remittance		reportsService, s3Service	auth, reports
Accounts > Petty Cash	5	accountingService, mastersService, pettyCashService, userService	accounting, masters, payments, users
Accounts > Bank Reconciliation	7	periodEndService, mastersService, reportsService, bankReconciliationService	accounting, clients, masters, reports, bank-reconciliation
Accounts > Insurer Reconciliation	1	insurerReconciliationService, remittanceService	insurer-reconciliation, masters
Accounts > Tax	13	birTaxService, periodEndService, mastersService, reportsService	bir, accounting, clients, masters, reports
Accounts > Period End	5	periodEndService	period-end, accounting
Accounts > Incentive	4	incentiveService	incentive, settings
Commission	4	commissionService, birTaxService, remittanceService	commission, insurer-overrides, masters, bir
Reinsurance	6	reinsuranceService, distributionService, mastersService	reinsurance, masters
Compliance	10	amlService	aml, uploads
Reports	21	mastersService, periodEndService, reportsService, distributionService	accounting, clients, masters, reports, claims
Master > Organization	2	mastersService	masters, uploads
Master > Insurance Management	12	mastersService, opsAccountingService, brandingService, reinsuranceService	masters, ops-masters, e-signatures, reinsurance, channels
Master > Location	3	mastersService	masters
Master > Employee Management	2	mastersService	masters
Master > User Management	8	accessControlService, mastersService, userService	access-control, masters, users
Master > Finance	25	mastersService, postingRulesService, packagesService, placementService	masters, accounting, posting-rules, placement, premium-charges
Master > System Configuration	9	integrationsService, adminService, brandingService, mastersService	masters, integrations, settings, branding, e-signatures
Master > Data Privacy	2	privacyService	privacy
Master > Go-Live and Data	1	goLiveDataService	data-load
Product Configurator	9	productConfiguratorService, mastersService, quotationService	product-configurator, masters, premium-charges

10.3 Prerequisites per back-end module

A screen works only when the data its module reads exists. The workbook repeats the prerequisites on every screen row, together with the master types the screen reads. The main ones:

Back-end module	Must exist first
quotations	Products and Insurance Company masters; coverages and vehicle masters for motor; motor tariff (seeded); Premium Taxes & LGU Rates; Commission Rate Matrix; a governing product template with its rules; a prospect or client
placement, comparison-reports	Client or prospect; Insurance Company and Products masters; numbering series for broker slips and placements
policies, cover-notes	Accepted quotation or bound placement; insurer credit terms; posting rules and account roles; open accounting period; for juridical clients the AML onboarding
endorsements, cancellations, fleet, marine	An issued policy; Short-Period Rates and Cancellation Reasons masters for cancellations
claims, claim-documents, motor-claims	An issued policy; claim settings; Claim Document Requirement and Repair Shop masters; signatories
renewals	Policies near expiry; renewals.* settings; jobs renewal-queue, renewal-pipeline and renewal-notices
receipts, pdc, collections, credit-control	Open bills; bank accounts; posting rules; open period; jobs receivable-ageing, collection-reminders, pdc-deposit-due
disbursements, payables, fixed-assets	Payables or supplier invoices; Supplier and Asset Class masters; bank accounts and cheque books; posting rules; open period
remittance, insurer-reconciliation, insurer-overrides	Collected premiums due to insurers; Remittance Master; Insurer Statement Formats; override agreements
commission, commission-rates, incentive	Commission Rate Matrix; referrers; issued policies with commission; Incentive Programs
reinsurance	Reinsurers and treaties; policies and claims to cede; cedants for facultative placement
period-end, bir	Fiscal years and periods; chart of accounts; tax codes; company legal identity; bir.* and invoice.* settings
aml	Clients; screening lists loaded; aml.* thresholds; Compliance Officer role granted
integrations	Connectors in test mode (seeded); provider credentials in the environment for live mode; COC series; bank file layouts
leads, channels, motor-programmes, campaigns	Numbering series; Philippine address masters; distribution channels; marketing consents for campaigns
documents, branding, e-signatures	Company master (letterhead and logo); signatories with signatures; document signature mapping
notifications	SMTP_URL in the environment and notification.email_enabled for e-mail delivery

10.4 Set-up order used by the UAT scenario

The UAT scenario script loads a new database in the order below before it creates any business transaction (backend/scripts/uat/phases/setup.js). The configuration kit of the go-live workbench follows the same principle: each sheet reads what the sheets before it created.

1. Persona users with their roles.
2. Products master.
3. Insurers with credit terms.
4. Commission Rate Matrix.
5. Bank accounts linked to the ledger.
6. Insurer Statement Formats.
7. Tax codes of the commission taxes (checked).
8. Sub-agent referrers.
9. Settlement limit of the Remittance Master.

10.5 Before enhancing a module

1. Find the screen in the workbook (sheet Screen dependencies) and note its front-end file, service file, endpoints and back-end modules.
2. Read the module's entry in the module catalogue, then its router.js for the endpoints, their permission and validation schema, then the service functions they call.
3. Check the other screens that call the same endpoints (filter the Endpoints column): a change in the response shape affects all of them.
4. Check the prerequisites column: a new mandatory field may need data in a master or a new setting first.
5. Check the scheduled jobs and posting events the module uses; a change in amounts reaches the ledger through posting rules.

11. API catalogue summary

11.1 Endpoints per module

1,291 endpoints are registered on the branch: 598 GET, 516 POST, 122 PUT, 9 PATCH and 46 DELETE. The table shows the count per back-end module and the number of menu screens that call it. The workbook sheet **APIs** lists each endpoint with its summary, permission and screen.

Back-end module	Base path(s) under /api	Endpoints	Menu screens
access-control	/access-control	24	7
accounting	/accounting	27	51
addresses	/addresses	9	2
aml	/aml	58	11
audit	/audit	1	0
auth	/auth	15	3
bank-reconciliation	/bank-reconciliation	38	4
bir	/bir	39	8
branding	/branding	11	1
campaigns	/campaigns	17	1
cancellations	/cancellations	3	1
channels	/channels	6	4
claim-documents	/claim-documents	6	1
claim-payments	/claim-payments	6	1
claims	/claims	17	5
clients	/clients, /customers	8	40
collections	/collections	9	1
commission	/commission	19	4
commission-rates	/commission-rates	7	1
comparison-reports	/comparison-reports	8	1
cover-notes	/cover-notes	7	1
credit-control	/credit-control	20	4
dashboard	/agent, /dashboard	6	5
data-load	/data-load	14	1
disbursements	/disbursements	15	2
document-numbering	/document-numbering	5	1
documents	/document-templates	8	5
e-signatures	/e-signatures	7	2
endorsements	/endorsements	9	3
fixed-assets	/fixed-assets	7	3
fleet	/fleet	14	1
incentive	/incentive	22	5
insurer-overrides	/insurer-overrides	15	2
insurer-reconciliation	/insurer-reconciliation	18	2
integrations	/bank-payments, /ctpl, /insurer-integration, /integrations, /messaging, /public	57	7
journal-vouchers	/journal-vouchers	9	3
leads	/lead, /lead-assignment, /leads	21	5
marine	/marine	17	1
masters	/masters	13	101
motor-claims	/motor-claims	10	1
motor-programmes	/motor-programmes	12	1
my-work	/my-work	14	1
notifications	/email, /notifications	11	5
ops-masters	/ops-masters	4	9

Back-end module	Base path(s) under /api	Endpoints	Menu screens
packages	/packages	27	3
payables	/payables	14	3
payment-gateway	/payment-gateways, /payment-links, /public	13	1
payments	/open-items, /payments, /petty-cash	29	7
pdv	/pdv	10	1
period-end	/period-end	45	8
placement	/broker-slips, /placements	26	10
policies	/policies	12	7
posting-rules	/account-determination, /posting-rules	22	4
premium-charges	/premium-charges	9	14
privacy	/privacy	14	3
product-configurator	/product-configurator	90	11
quotations	/biCoverage, /email, /master, /paCoverage, /pdCoverage, /quotations, /quote	43	5
receipts	/billing-statement, /receipts	19	3
reinsurance	/reinsurance	51	7
remittance	/remittance	86	17
renewals	/policy-renewals, /renewals	47	11
report-builder	/report-builder	11	1
reports	/reports	13	32
schedules	/schedules	4	1
search	/search	1	0
settings	/settings	6	14
system	/version	1	0
system-settings	/system-settings	7	2
uploads	/s3, /upload	11	6
users	/roles, /users	17	8

The menu screen count of shared modules (masters, accounting, clients, settings, reports, premium charges) is high because many screens read masters, settings and account lists or upload files. The two health endpoints (GET /api/health, GET /api/health/live) are mounted in src/app.js outside the registry. Package B adds ic-compliance (38) and data-breaches (11); package G adds sales-activities (8) and 8 endpoints to fixed-assets, period-end and product-configurator.

11.2 Public endpoints

These endpoints answer without a bearer token. Each has its own control.

Endpoint	Control
POST /api/auth/login, /login/2fa, /refresh, /logout, /forgot-password, /reset-password; GET /api/auth/password-policy	Sign-in rate limit, logout, hashed reset codes, refresh rotation
GET /api/s3/object/*, GET /api/upload/file/*	Bearer token or a valid signed link
GET /api/reports/generated/:id/download	Bearer token with report access, or a signed download token
POST /api/quotations/approve-by-customer	Signed approval token from the e-mail link
GET /api/public/payments/:token, /policy.pdf, POST /sandbox	Signed payment-link token; the simulation works only on sandbox payment links
GET, POST /api/public/payments/webhooks/:gateway	Gateway signature checked on the raw body
POST /api/public/integrations/inbound/:connector	HMAC-SHA256 signature of the raw body with the connector's webhook secret; connector enabled and integrations.inbound_enabled; unsigned messages are kept as ignored, never applied

Endpoint	Control
GET, POST /api/campaigns/opt-out/:token	Signed opt-out token of one recipient; valid campaigns.opt_out_link_days
GET /api/branding, GET /api/branding/assets/:name	Branding payload and images for the sign-in page only (with ETag)
GET /api/settings/public, GET /api/system-settings, GET /api/version	Branding and version information only

11.3 Keeping the API documentation current

npm run export:api in backend/ loads every module and writes to backend/docs/api/:openapi.json (OpenAPI 3.1), the Postman collection and environment, BrokerVerse_API_Touchpoints.xlsx (API List, By Screen, Modules) and a CSV. The Postman collection signs in first and keeps the token in {{token}}. Run it after every change to a router and commit the result with the change; then regenerate the dependency workbook with python3 docs/package/tools/build_api_catalogue.py.

12. How to review a change

This chapter is the reviewer's guide for a pull request into `brokerverse-platform`. A change is approved when it follows the conventions of this document, its tests and checks pass, and the documentation that its users read is updated with it.

12.1 What the pull request must contain

- One topic per pull request, with a description that names the screens, endpoints, tables, settings, jobs and posting events it adds or changes, and the migration numbers it uses.
- The code, its migration, its seed data, its tests, the translations, the menu entry and grants, the help entry, the API documentation (`npm run export:api`) and the documentation source it affects (`docs/package/source/*.md`: user manual, schedules and batch jobs, compliance matrix, technical reference).
- No generated or local files: no `node_modules`, `.env`, uploaded files, database dumps or build output.

12.2 Code standards to check

Area	The reviewer checks
Routes	Declared with <code>define()</code> , with summary, screen (menu path), example request and response; <code>requirePermission()</code> or an administrator role, or deliberately public with its own control (token, signature)
Validation	Body and query validated with <code>zod</code> ; business rules checked in the service with the <code>lib/errors.js</code> helpers
SQL	<code>\$n</code> parameters only; dynamic identifiers from a fixed map or allow-list; writes to more than one table in <code>withTransaction()</code> ; rows changed under concurrency read <code>FOR UPDATE</code> ; services take <code>db</code> as first argument
Money and dates	Amounts rounded with <code>lib/money.js</code> ; ledger amounts in the base currency; "today" from <code>lib/dates.js</code> (business time zone), never <code>new Date()</code> or <code>current_date</code> for a business date
Posting	Journals only through <code>postEvent()</code> with a seeded rule using account roles; no hand-built journal lines
Numbers	Document numbers only through <code>nextDocumentNumber()</code> ; a new series seeded with a unique prefix
Audit	Every change calls <code>audit()</code> with before and after images; secrets never in the images
Settings	Business values read with <code>getSetting()</code> ; new keys seeded with group and label; <code>npm run check:settings</code> passes
Secrets	None in code, seeds or tests; new environment variables in <code>deploy/backend.env.example</code> and <code>deploy/REFERENCE.md</code> ; production refuses placeholders
Logging	No <code>console.log</code> ; <code>req.log</code> or <code>logger</code> ; nothing personal or secret in a log line
Personal data	New personal columns classified in the personal data catalogue; identifiers that must be encrypted added to <code>PII_STORAGE</code> (package B)
Front end	Calls only through a service file; texts in <code>en.json</code> ; menu entry, grants and help entry added; theme tokens, not literal colours; list and form patterns of the existing screens

12.3 Test expectations

- **Back end.** Every new endpoint and every changed rule is covered in `backend/test/<module>.test.js` through the HTTP API with `supertest`: the happy path, the permission (403 for a role without it), the validation (400) and

each business refusal (409 or 400 with the message the user sees). A posting change asserts the journal lines. A job asserts its summary and that a second run on the same day changes nothing. Shared helpers are in `test/helpers.js` and `test/accounting.fixtures.js`.

- **Guard tests that fail when a convention is missed:** `reset-transactions.test.js` (every table classified), `mask-data.test.js` (every personal column in the catalogue), `hardening.test.js` (every non-public route answers 401 without a token), `role-access.test.js` and `user-access.test.js` (role grants), `settings-ownership.test.js` (settings owned by one screen), `dependencies.test.js`.
- **Front end.** Logic that decides what a user sees (menu, permissions, rules, formatting) has a jest test next to it; a new menu entry extends `utils/menuPermissions.test.js` or the area's screen test; `helpRoutes.test.js` passes.
- **End to end.** A change to a business flow is run through `backend/scripts/uat-scenario.js` on a new database before it goes to UAT.

12.4 Table classification and personal data catalogue

- Every table a migration creates is added to exactly one list of `backend/scripts/lib/table-classification.js`: `TRANSACTION_TABLES` (emptied by the transaction reset), `SYSTEM_TABLES` with its reset action, or `MASTER_CONFIG_TABLES` (kept). A transaction table referenced by a master table needs its foreign keys checked. `test/reset-transactions.test.js` fails otherwise.
- Every column that holds personal data, or whose name looks personal (`PERSONAL_NAME_RE`), is added to `CATALOGUE` of `backend/scripts/lib/pii-catalogue.js` with its masking rule, or to `ALLOW_LIST` with the reason it is kept. `test/mask-data.test.js` fails otherwise. The same catalogue drives masking by role and the data dictionary's personal data classification.
- The table also goes into `docs/architecture/tools/table_catalog.py` (domain, owner module, purpose, retention class) and, for the data dictionary, into an area of `docs/package/tools/data-dictionary/tables_meta.py`.

12.5 Migration numbering

- File name `NNNN_short_name.sql`, the next free number in the range agreed for the change. Parallel work packages reserve a range so that they do not collide (for example package B uses 0270 to 0277 and package G 0320 to 0329); within a range numbers may leave gaps.
- A migration is safe on a database that already has data and on a new one: `IF NOT EXISTS, ON CONFLICT DO NOTHING`, defaults for new mandatory columns, data conversions with a `WHERE` clause. It starts with a comment saying what it adds and which screen uses it.
- Unique values seeded by a migration (document numbering prefixes, permission codes, master type codes, job codes, posting event codes) are checked against those already in use on the branch and in the packages

developed in parallel. Example found while writing issue 1.1: migration 0272_complaints_register.sql of package B seeded the series prefix CMP, already used by the comparison report series of 0306; the unique index document_numbering_active_prefix would have stopped the migration on a database that has both. The prefix was changed to CPT before the merge.

- Never edit a migration that has left a developer's machine; a correction is a new file.

12.6 Translations, menu, grants and help

1. Texts: every new key in brokerverse/src/locales/en.json, grouped under the screen's key; `npm run check:i18n` lists keys used in code and missing.
2. Menu: the entry in components/SideBar/list.js (name, path, includes for related pages), in the section where users expect it (Master in sections).
3. Route: routes/MainRoute.js, wrapped by ProtectedRoute.
4. Grants: the entry in roleMenuPermissions of utils/menuPermissions.js for each role that holds the permission of the screen's endpoints; the permission itself granted in the migration and in ROLE_PERMS of seed.js.
5. Help: an entry in components/HelpPanel/helpRoutes.js pointing to the user manual heading; `npm run help:build` after the manual is updated.

12.7 Lint and checks to run

Where	Command	Expected
backend/	<code>npm run lint</code>	no errors, no warnings
backend/	<code>npm test</code> (needs TEST_DATABASE_URL or the database brokerverse_test)	all tests pass
backend/	<code>npm run check:settings</code> (with DATABASE_URL of a migrated and seeded database)	0 keys read in code and missing
backend/	<code>npm run export:api</code>	no module skipped; commit backend/docs/api
brokerverse/	<code>npm run lint</code>	no errors; no new warnings
brokerverse/	<code>CI=true npm test -- --watchAll=false</code>	all suites pass
brokerverse/	<code>npm run check:api, npm run check:i18n</code>	0 calls to missing routes; no new missing keys
docs/package/tools	<code>python3 build_all.py <changed sources></code>	documents build; pages checked visually

12.8 CI gates

The four required status checks of the branch (Backend lint and tests, Front-end lint, tests and build, Backend release artefact, Dependency audit) must be green, and one reviewer must approve. Deployment to dev follows the merge automatically; UAT and Production need their tags and approvals (chapter Platform engines, "Release pipeline").

13. Guide for the production support team

13.1 Running locally

1. Install Node.js 22 and PostgreSQL 16; create the database brokerverse (or run `node scripts/create-database.js`).
2. In `backend/`: copy `.env.example` to `.env`, run `npm ci`, then `npm run dev`. The first start applies the migrations and the seed; the log prints the administrator password when `ADMIN_PASSWORD` is not set.
3. In `brokerverse/`: run `npm ci --legacy-peer-deps`, then `REACT_APP_BASE_URL=http://localhost:8000/api npm start (port 3000)`.
4. Sign in as BrokerVerse. With `SEED_SAMPLE_DATA` on (the development default) demo data is loaded.
5. Before a change: the checks of the chapter "How to review a change".

`docker compose up --build` at the repository root starts PostgreSQL, the API and the web application together on port 8080.

13.2 Where the logs are

Installation	API log	Other records
EC2 with PM2 (<code>deploy/ec2</code>)	<code>pm2 logs brokerverse-api</code> ; files under the PM2 home of root (<code>~/ .pm2/logs/</code>)	Database tables below
Docker Compose	<code>docker compose logs api</code>	Database tables below
Local development	The terminal running <code>npm run dev</code>	Database tables below

Database records that support the logs: `audit_log` (changes, with source), `login_history` (sign-ins), `job_runs` (scheduled jobs), `email_outbox` (e-mails and their errors), `integration_outbox` and `integration_attempts` (third-party calls), `eis_submissions`, `claim_history`, `period_status_history`, `bank_reconciliation_history`. Health: GET `/api/health` (readiness, database and pending migrations), GET `/api/health/live`, GET `/api/version` (commit and build time).

13.3 Tracing a defect from a screen to the SQL

1. **Get the request id.** The user quotes it from the error message, or the support analyst reads the `x-request-id` response header in the browser's network tab.
2. **Find the log line.** Search the API log for the request id. The line holds the method, URL, status, response time and, for a 5xx, the error message and stack.
3. **Find the route.** Look up the method and path in the workbook sheet APIs, or search `backend/src/modules` for the path in a `define()` call. The screen field confirms the screen; the module catalogue gives the module's tables and jobs.

4. **Find the service and SQL.** The handler calls a function of the module's service file; the SQL is in that function. Run the same statement in psql with the parameters from the request to reproduce.
5. **Check the history.** The History panel of the record, or Master > System Configuration > Audit Trail, shows who changed it, when, from which screen and which fields; journals point back to their source through `journal_vouchers.reference_type` and `reference_id`.

13.4 Common failure patterns and where to look

Symptom on screen	Likely cause	Where to look
"Invalid username or password", then "Account locked"	Wrong passwords reached <code>limits.max_login_attempts</code>	<code>login_history</code> ; unlock on Master > User Management
429 "Too many requests"	Sign-in or API rate limit	<code>security.login_rate_limit</code> , <code>security.api_rate_limit</code> ; wait for Retry-After
"Session ended; sign in again"	Token version raised (password reset, role change, deactivation)	<code>users.token_version</code> ; expected behaviour
403 "Requires permission: write:..."	Role lacks the permission	Master > User Management > Role Permissions
Screen missing from the menu	Role not granted the menu entry	<code>roleMenuPermissions</code> in <code>utils/menuPermissions.js</code>
"Accounting period ... is closed / soft-closed / locked"	Posting date in a closed period	Accounts > Period End > Period Management
"No active posting rule for event ..." or "GL account setting ... is not configured"	Posting configuration incomplete	Master > Finance > Posting Rules and Account Determination
"Document numbering series ... is not configured"	Numbering series missing or inactive	Master > Document Numbering
Policy issue refused for a juridical client	AML onboarding or EDD not complete	Compliance > Client Due Diligence and EDD Reviews; <code>aml.block_issue_pending_edd</code>
Referrer payout refused (package B)	Referrer licence lapsed in the licence register	Compliance > Insurance Commission > Licence Register
TIN or bank account shows as masked (package B)	User lacks <code>view:pii</code> , or reveal mode is on request	Role Permissions; <code>privacy.pii_reveal_mode</code>
E-mails not received	<code>notification.email_enabled</code> off, <code>SMTP_URL</code> empty or SMTP error	<code>email_outbox.status</code> and error; Master > E-mail Outbox
SMS, CTPL or insurer message not sent	Connector in test mode, disabled, credential variable missing, or retries exhausted	Master > System Configuration > Integrations (monitor); <code>integration_attempts</code>
A scheduled job did not run	Job disabled (many are delivered off), scheduler off on the instance, or job error	Master > System Configuration > Schedules; <code>job_runs</code> ; <code>SCHEDULER_ENABLED</code>
Go-live load refused	Validation errors in the workbook	Download the errors workbook on the Go-Live Data Load screen
Transaction reset or masking tool refuses	Go-live lock on, database marked production, or a table not classified	<code>golive.locked</code> , <code>system.environment</code> , <code>table-classification.js</code>
File link answers "This file link has expired"	Signed link older than <code>FILE_URL_TTL_SECONDS</code>	Reload the screen to get a new link
413 or 415 on upload	File too large or type not allowed	<code>UPLOAD_MAX_MB</code> , <code>uploads.allowed_types</code>
API does not start in	Unsafe configuration	First log line names the

Symptom on screen	Likely cause	Where to look
production		variable (JWT_SECRET, PII_ENCRYPTION_KEY, CORS_ORIGINS ...)
/api/health answers 503	Database unreachable or migrations pending	Log lines applied <file> or migration <file> failed

13.5 Making a change safely

Add a field to a screen.

1. Add the column in a new migration (ALTER TABLE ... ADD COLUMN IF NOT EXISTS); classify it in the personal data catalogue if it holds personal data.
2. Add it to the service: the field map for inserts and updates, and the row mapper for responses.
3. Add it to the zod schema and the example in router.js; run npm run export:api.
4. Add the input to the screen, the label to locales/en.json, and the value to the service call.
5. Add or extend a test in backend/test/<module>.test.js; add the field to the go-live workbook sheet if it is loaded at go-live.

Add a screen.

1. Create the component under module/<Area>/ (back office) or agentModule/<area>/ (operations).
2. Add the route in routes/MainRoute.js and the menu entry in components/SideBar/list.js (path and includes).
3. Grant the entry to the roles in utils/menuPermissions.js; extend utils/menuPermissions.test.js.
4. Add the help entry in components/HelpPanel/helpRoutes.js and the section in the user manual.
5. Call the API only through a service file; check that the roles hold the permissions the endpoints need.

Add a report. Add a base query in backend/src/modules/reports/queries.js, a report_definitions row in a migration or seed (code, name, category, columns, permission), and a test; see backend/src/modules/reports/README.md. For a user-defined listing, add a dataset to report-builder/datasets.js instead.

Add a setting. Insert the key with group, label, type and value in a seed or migration with ON CONFLICT (key) DO NOTHING; read it with getSetting('group.key', <seeded value>); run npm run check:settings.

Add a migration. See "Migration numbering". Add every new table to the table classification and the catalogues.

Add a scheduled job. Write the handler in the module's jobs.js, re-export it from backend/src/jobs/handlers.js, and seed the job in its migration or in backend/src/db/seeds/jobs.json (code, name, cron, handler, parameters, enabled);

seed it off when it needs a decision or a provider, and add it to the schedules and batch jobs document.

Add an integration. Follow "Integration framework": adapter, message types, connector row in test mode with `credential_env`, `enqueue()` inside the business transaction, tests with the fake provider.

13.6 Release and rollback

Releases follow `deploy/RELEASE_PIPELINE.md` (summary in the chapter Platform engines): merge, release candidate tag to UAT, release tag to Production, each with its approval, pre-deploy backup, forward-only migration, smoke test and automatic rollback of the application. The checks after a deployment are: `/api/health` ready, `/api/version` shows the deployed commit, sign-in, Master > System Settings and Master > Configuration load, Master > Schedules lists the jobs, the Integrations monitor shows no stuck messages.

Rollback:

- Application: Actions > Rollback to the earlier tag; no migration is undone, because migrations only add and the previous release runs on the newer schema.
- Data: restore the pre-deploy backup only when data written by the new release must be removed; decide with the broker's CAB, because transactions entered since the backup must be entered again.

14. Document Sign-Off

Serial Number	Description	Owner	Status
1	No open items at the time of issue	iorta TechNXT	

Sign Off Date	Signature	Approved By	Department & Project Role



Insurance technology for the Philippines



10+

COUNTRIES

Global Presence



8

GLOBAL OFFICES

Strategically Located



11+

MARKETS SERVED

Across Continents



100%

COMMITMENT

To Excellence



MALAYSIA



INDIA



PHILIPPINES



THAILAND



VIETNAM



USA



ETHIOPIA



CAMBODIA



HONGKONG



SINGAPORE



Scan More Details